

TGR-LORA: A CONTINUAL LEARNING PIPELINE FOR SMALL LANGUAGE MODELS

A Design Project Report

Presented to the School of Electrical and Computer Engineering of Cornell
University

in Partial Fulfillment of the Requirements for the Degree of
Master of Engineering, Electrical and Computer Engineering

Submitted by

Beichen Yu (by325)

MEng Field Advisor: Adit Jain (aj457@cornell.edu), Vikram Krishnamurthy
(vikramk@cornell.edu)

Degree Date: December 2026

Abstract

Master of Engineering Program
School of Electrical and Computer Engineering
Cornell University
Design Project Report

Project Title: TGR-LoRA: A CONTINUAL LEARNING PIPELINE FOR SMALL LANGUAGE MODELS

Author: Beichen Yu (by325)

Abstract:

This project implements and evaluates a parameter-efficient (PEFT) continual learning (CL) method designed for small language models (SLMs). It aims to address the problem of catastrophic forgetting, which means during training process across a sequence of different tasks, the parameter changes for learning later tasks might degrade the model’s performance on previous tasks. Our project utilizes the Qwen2.5-3B-Instruct model and conducts experiments using five subsets from the TRACE continual learning enchmark, including C-STANCE, FOMC, ScienceQA, NumGLUE-cm, and NumGLUE-ds.

We implemented two distinct methods. Sequential LoRA serves as our baseline. It freezes the base model parameters and trains a single LoRA adapter across all tasks.

TGR/OrthoHist-LoRA is our method, which trains an active LoRA branch for the current task while integrating and consolidating the updated parameters into a frozen pool of historical bases and coefficients. In order to effectively use historical information, it also learns a low-rank mixing operator (defined as $M_t = I + U_t V_t^\top$) for the current task, thereby enabling the reuse of historical activation information. Furthermore, we implemented the metrics for evaluating continual learning performance, visualizations such as heatmaps showing knowledge reuse degree.

Executive Summary

The main objective of our project was to mitigate catastrophic forgetting while enabling subsequent tasks to leverage useful information acquired from prior tasks. Our system is entirely software-based. The core components include the Qwen2.5-3B-Instruct model, LoRA-based PEFT methods, TGR-LoRA method, and the TRACE benchmark.

Our proposed method involves **training an Active LoRA branch** tailored to the current task. Updated parameters from completed tasks **are stored in a frozen historical pool**, and each new task is equipped with a trainable low-rank mixing operator for reusing these historical activations. During the training process, both the base model and the historical pool remain frozen. There are 2 benefits: preventing the new training process from directly overwriting previously learned directions, while allowing subsequent tasks to reuse knowledge acquired during earlier tasks at the same time. In summary, the design philosophy is that the Active LoRA branch facilitates the model’s exploration of new knowledge of the current task, while the low-rank mixing operator enables the efficient reuse of historical knowledge.

We successfully implemented a comprehensive, task-by-task training loop with checkpoint-based resume, task-level JSON logging, score matrix generation, Backward Transfer (BWT) calculation, average accuracy (AA) computation, and visualizations such as feature reuse heatmaps for the historical mixing operators.

During the project’s initial poster presentation phase, reported results indicated more pronounced positive BWT scores than those subsequently reproduced in the final implementation. During the final code integration and validation phase, we refined our evaluation method, using to a method that computes task-by-task score matrices and BWT metrics. Under this more rigorous evaluation standard, we further refined our experimental results and drew the following conclusions:

....

Individual Contributions

Beichen Yu (by325): This is an individual project. I was responsible for the project’s overall design, code implementation, debugging, experimental execution, poster creation, code repository organization, and final report writing. My specific contributions include:

- Defined the Continual Learning problem setting, selecting the Qwen2.5-3B-Instruct model and the TRACE benchmark as the foundation for experimental validation.
- Implemented a Sequential LoRA baseline model using the PEFT library, freezing the parameters of the Base Model during the implementation process.
- Implemented the TGR-LoRA layer wrapper module. The components include the frozen base model, an active LoRA branch, a frozen pool of historical basis and coefficients, an orthogonal gradient projection mechanism, and a low-rank mixer $M_t = I + U_t V_t^\top$, etc.
- Constructed a Task-by-Task Trainer, checkpoint saving logic, training resume logic, JSON logging function, a score matrix saving function, and a module for analyzing historical knowledge reuse.
- Implemented task-specific answer parsing functions and text generation evaluation metrics for five datasets: C-STANCE, FOMC, ScienceQA, NumGLUE-cm, and NumGLUE-ds to ensure an accurate evaluation method.
- Executed and debugged various experiments within Colab and AutoDL GPU environments, resolving technical challenges such as PyTorch/Transformers compatibility issues, CUDA architecture adaptation problems, and RTX 4090 GPU memory limitations.
- Organized and published the final GitHub code repository including the README and configuration files, and made detailed documentation to ensure the reproducibility of the experiments.

Contents

Abstract	i
Executive Summary	ii
Individual Contributions	iii
1 Problem Statement and Requirements	1
1.1 Design Problem	1
1.2 Requirements	1
1.3 Practical Constraints	2
1.4 Final Design	3
2 Review of Alternatives and Selected Approach	4
2.1 Full Fine-Tuning	4
2.2 Sequential LoRA	4
2.3 Adapter-per-Task	5
2.4 Fisher-SVD Design	5
2.5 Final Selected Approach: TGR/OrthoHist-LoRA	5
3 System Design and Implementation	9
3.1 System Overview	9
3.2 Repository Structure	9
3.3 Sequential LoRA Baseline Implementation	10
3.4 TGR-LoRA Implementation	10
4 Testing and Experimental Validation	12
4.1 Evaluation Method	12
4.2 Sequential LoRA Five-Task Baseline	13
4.3 TGR-LoRA Five-Task Comparison	13
4.3.1 TGR-LoRA with $mt_k = 4$	14
4.3.2 TGR-LoRA with $mt_k = 8$	14
4.3.3 TGR-LoRA with $mt_k = 16$	15
4.3.4 Final Method Comparison	15

5	Reproducibility, User Manual, and Bill of Materials	17
5.1	Code Repository	17
5.2	Environment Setup	17
5.3	Dataset Layout	17
5.4	Running Experiments	18
5.5	Expected Outputs	18
5.6	Software Bill of Materials	19
A	Appendix A: Code Listings	21
A.1	Project Entry Point	21
A.2	Experiment Runner	23
A.3	Result Summary Script	24
A.4	Python Requirements	27
A.5	Configuration: Sequential LoRA	27
A.6	Configuration: OrthoHist-LoRA k=4	28
A.7	Configuration: OrthoHist-LoRA k=8	29
A.8	Configuration: OrthoHist-LoRA k=16	30
A.9	Configuration Loader	31
A.10	TRACE Data Loader	32
A.11	TRACE Evaluation Utilities	36
A.12	Training Loops	45
A.13	OrthoHist-LoRA Implementation	58
A.14	Checkpointing Utilities	63
A.15	Analysis and Visualization Utilities	64
B	Appendix B: User Manual	67
B.1	Quick Start	67
B.2	Reading Final Metrics	67

Chapter 1

Problem Statement and Requirements

1.1 Design Problem

Continual Learning aims to enable models to train on a continuous stream of incoming tasks. While learning a current task, a model must maintain its performance on previously learned tasks without degradation. Large Language Models (LLMs) are very susceptible to this challenge, as gradient updates applied to subsequent tasks may overwrite specific directions of model parameters that are critical for earlier tasks. The direct consequence of that is Catastrophic Forgetting.

This project focuses on PEFT supervised fine-tuning methods and Small Language Models (SLMs), as these methods are more lightweight and easier to deploy in real-world scenarios. We selected LoRA, as it achieves excellent results by freezing the base model and training only a set of low-rank adapter matrices. However, if a single LoRA adapter is trained sequentially across tasks, the model may still struggle to avoid the forgetting problem. While training a separate adapter for each task effectively prevents interference between tasks, this approach suffers from extremely low parameter reuse. Furthermore, during the inference phase, the corresponding adapter must be manually selected for each specific task, which can be complex to manage. Consequently, the core design challenge of this project is constructing a dynamically growing continual learning system that can both effectively preserve information of historical tasks and enable the efficient reuse of that information.

1.2 Requirements

The final requirements evolved iteratively over the two semesters. The focus of the first semester was to investigate the causes of catastrophic forgetting. Through a comprehensive literature review, we knew that catastrophic forgetting is most severe when the input spaces are similar but the output spaces differ. The work in the second semester focused on designing a robust training and evaluation pipeline with mathematical rigor. The finalized requirements are as follows:

1. **Sequential Task Training:** The system must train tasks sequentially in a fixed order, performing an evaluation after the completion of each task.
2. **Parameter Efficiency:** The Base Model must remain frozen, and only parameters associated with LoRA or TGR-LoRA are permitted to be trained.
3. **Baseline Comparison:** The system must employ Sequential LoRA as a fair comparative baseline.
4. **Historical Memory:** Update information (learned knowledge) generated by completed tasks must be stored within a fixed structure (served as memory).
5. **Historical Reuse:** Subsequent tasks should be able to reuse historical low-rank activations generated by previous tasks via a trainable mixing operator.
6. **Forgetting Metrics:** The system must compute and output a score matrix, average accuracy (AA), and the Backward Transfer (BWT) metric.
7. **Checkpoint Recovery:** If there is an interruption to the Colab or cloud GPU runtime, the training process must be capable of resuming from a task-level checkpoint.
8. **Reproducibility:** The project repository and report must include the complete codebase, execution commands, configuration files, explanations of output results, and environment setup instructions.
9. **Forgetting Mitigation:** The proposed method should effectively reduce the degree of forgetting and achieve a measurable level of Backward Transfer (BWT).

1.3 Practical Constraints

The project faced several constraints, these factors ultimately shaped our design approach.

- **GPU Memory.** The Qwen2.5-3B-Instruct model is substantial in size. Consequently, when performing TGR-LoRA training on an RTX 4090 card with a sequence length of 1024 (model generation) and a historical pool that continuously expands throughout the training process, GPU memory usage can result in an Out of Memory (OOM) error.
- **Cloud Runtime Environment.** Constrained by the resource limits of the Colab and the dynamic variability of the AutoDL environment, as well as the long duration of the training process, the project necessitated the task-level checkpointing to ensure the robustness of the training workflow.
- **Software Compatibility.** Compatibility among PyTorch, Transformers, PEFT, CUDA, and the underlying GPU architecture directly impacted experiments implementation. For instance, the RTX 5090 card requires the use of newer versions of the CUDA and PyTorch software stacks, so we finally choose 4090 for simpler configuration.

- **Evaluation Diversity.** The five TRACE tasks involve distinct types of responses. Therefore, it was necessary to design task-tailored functions for result parsing and the calculation of evaluation metrics.
- **Scope Reduction.** Although the project poster referenced the complete TRACE benchmark dataset, constraints regarding computational resources and time costs ultimately reduced our scope, with the final implementation of five representative tasks.

1.4 Final Design

Design	Final implementation
Backbone model	Qwen2.5-3B-Instruct
Task sequence	C-STANCE, FOMC, ScienceQA, NumGLUE-cm, NumGLUE-ds
Baseline	Sequential LoRA
Proposed method	TGR/OrthoHist-LoRA
Historical memory	Frozen QR-based basis-coefficient pool
Historical reuse	$M_t = I + U_t V_t^\top$ low-rank mixer
Metrics	Task score, average accuracy, BWT, score matrix
Artifacts	<code>score_matrix.png</code> , model checkpoints

Table 1.1: Final design specifications.

Chapter 2

Review of Alternatives and Selected Approach

2.1 Full Fine-Tuning

Full fine-tuning updates all of the model parameters. While this achieves the maximum flexibility, it is computationally expensive and therefore unsuitable for this project. Not only does it necessitate the storage of massive checkpoint files, but is also highly prone to severe catastrophic forgetting. Furthermore, this runs counter to our objective of achieving parameter efficiency.

2.2 Sequential LoRA

LoRA adds a low-rank adaptation branch to the original linear layer. It represents a trainable update to a linear layer as:

$$\Delta W = BA,$$

where $A \in \mathbb{R}^{r \times d_{\text{in}}}$ and $B \in \mathbb{R}^{d_{\text{out}} \times r}$. The forward residual is:

$$\Delta h = BAx.$$

The model’s forward pass for a given input x is represented as the sum of the frozen base layer output and the low-rank adaptation:

$$h = W_0x + \Delta Wx = W_0x + \frac{\alpha}{r}BAx, \quad (2.1)$$

where $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is the frozen pre-trained weight matrix, and $\frac{\alpha}{r}$ is a scaling factor defined by the LoRA rank r and the hyperparameter α . During training, only A and B receive gradient updates, while W_0 is frozen.

Sequential LoRA trains one adapter across tasks. It is simple and parameter-efficient, so it is the main baseline. Its weakness is that later task gradients may overwrite the same

LoRA matrices/important directions used by earlier tasks.

2.3 Adapter-per-Task

A separate LoRA adapter could be saved for each specific task. While this method can effectively minimize interference between tasks, it also avoids the sharing of knowledge across them. Furthermore, during the inference phase, this approach needs manual selection of task-specific adapter, which increases the complexity of management.

2.4 Fisher-SVD Design

During the initial phase of design exploration, we designed a method combining Fisher-weighted rank selection with Singular Value Decomposition (SVD)-based basis extraction. Specifically, once a new task is trained, we first employ Fisher information to assess the significance of the active LoRA rank

$$R_j = b_j \otimes a_j^\top = b_j a_j^\top$$

associated with that task. Then, we would utilize Fisher-weighted SVD to compress these critical directions and store them within a historical memory module. Finally, during the training of subsequent tasks, these previously saved directions would be protected. However, this implementation does not achieve a lossless compression of the knowledge acquired during each task, and Fisher information can be accurately estimated only when the model has reached the optimal training state of current task.

2.5 Final Selected Approach: TGR/OrthoHist-LoRA

Our final implementation is designed to mitigate catastrophic forgetting while allowing controlled reuse of historical knowledge. It consists of the following components:

1. **Frozen Base Model:** The original Qwen2.5-3B-Instruct model is frozen. All low-rank adaptation occurs in separate trainable branches.
2. **Active LoRA Branch:** For each new task, a dedicated low-rank adapter is trained. The forward computation for a wrapped linear layer is:

$$h' = W_0 x + \frac{\alpha}{r} (\Delta h_{\text{active}} + \Delta h_{\text{history}})$$

where the active task contribution is

$$\Delta h_{\text{active}} = B_{\text{active}} A_{\text{active}} x.$$

The active branch captures task-specific updates without modifying the frozen base weights.

3. **Historical Basis-Coefficient Pool:** At the end of each task, the active update is committed into a frozen historical pool using QR factorization:

$$B_{\text{active}}A_{\text{active}} = Q(RA_{\text{active}}),$$

where Q (basis) and RA_{active} (coefficients) are stored as historical memory. The coefficients represent how input information is projected into the low-rank feature space (rank-dimension), while the basis maps these features back to the output dimension to modify the model response (output). Both Q and RA_{active} are frozen and can be reused in later tasks. We use QR decomposition to ensure that the model learns different directions/aspects of knowledge for a task, also laying the foundation for gradient orthogonal projection during the training of subsequent tasks.

4. **Low-Rank Historical Reuse:** For a new task t , all previous basis and coefficient blocks are concatenated:

$$B_{\text{old}} = [Q_1, Q_2, \dots, Q_{t-1}], \quad C_{\text{old}} = \begin{bmatrix} C_1 \\ C_2 \\ \vdots \\ C_{t-1} \end{bmatrix}.$$

A trainable low-rank mixing matrix $M_t = I + U_t V_t^\top$ is applied to flexibly combine historical contributions:

$$\Delta h_{\text{history}} = B_{\text{old}} M_t C_{\text{old}} x.$$

This allows the model to reuse previous task knowledge while keeping it separate from the current task updates. mt_k denotes the rank of the mixing matrix M_t , which determines the low-rank dimension for linearly combining historical bases from the Basis Pool. To ensure parameter efficiency, M_t is decomposed into two matrices: $M_{t,U} \in \mathbb{R}^{(r \cdot i) \times mt_k}$ and $M_{t,V} \in \mathbb{R}^{(r \cdot i) \times mt_k}$ (i represents the number of historical tasks). By constraining M_t to a lower rank, the model can combine those features that are most significant from past tasks and possess strong generalization capabilities.

5. **Orthogonal Gradient Hook/Projection:** To prevent interference between new task learning and previously stored historical directions, a backward hook is applied on the active LoRA branch. This hook projects gradients of the active branch onto the subspace orthogonal to the historical basis. So the model’s output direction of the new task is orthogonal to all historical tasks’ output directions. To be specific, if G_{active} is the gradient for the active LoRA B matrix, the hook computes:

$$G_{\text{active}} \leftarrow G_{\text{active}} - Q_{\text{old}}(Q_{\text{old}}^\top G_{\text{active}})$$

where Q_{old} is the concatenated basis from all previous tasks. This ensures that updates

for the current task do not overwrite directions important for historical tasks, effectively reducing catastrophic forgetting.

In summary, our design separates old-task memory from new-task residual learning, allows controlled low-rank reuse, and incorporates orthogonal gradient projection to preserve historical knowledge while adapting to new tasks. During the training process, the base model and historical pool are frozen, while the low-rank mixing matrix and active LoRA are trainable as shown in the following figure.

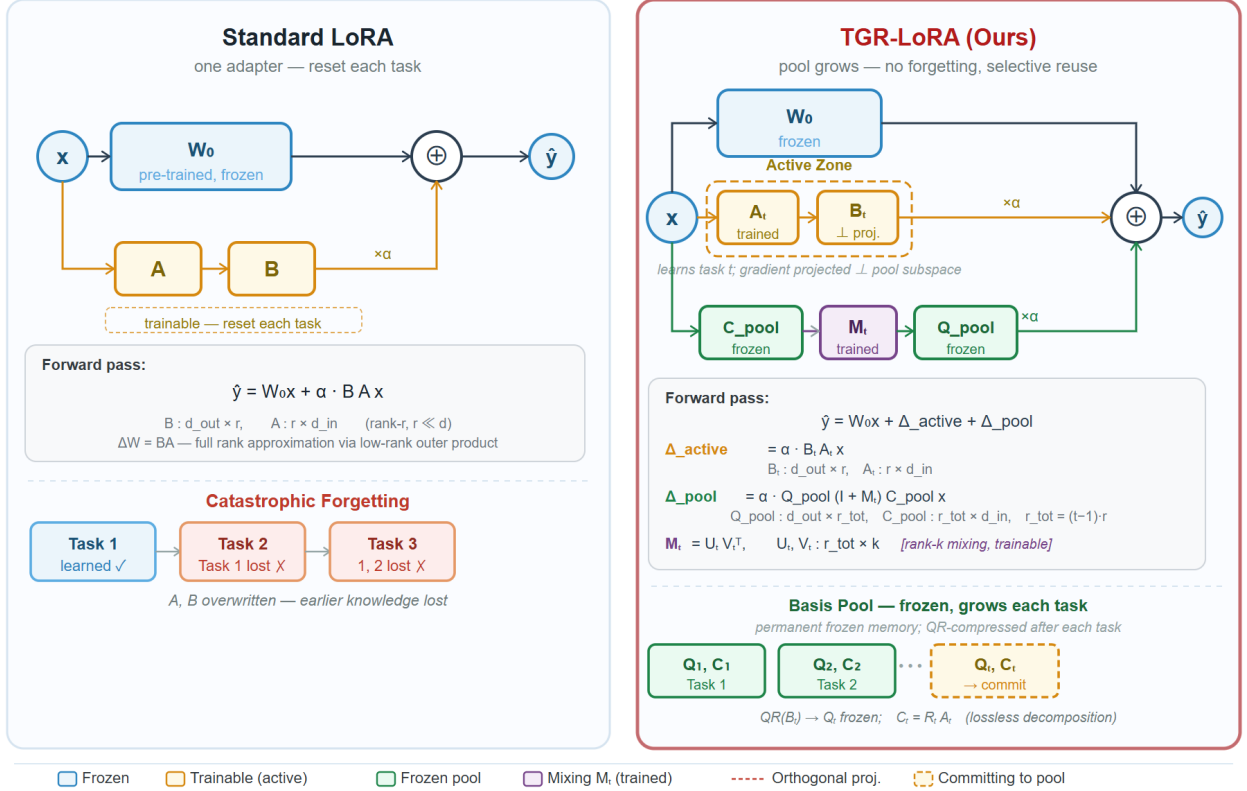


Figure 2.1: Final Design

TGR-LoRA Algorithm

Algorithm 1: TGR-LoRA Continual Learning

Input: Tasks $\{T_1, \dots, T_N\}$; LoRA rank r ; scaling α ; mt-rank k

Output: Model + frozen Basis Pool

```

for each task  $t = 1$  to  $N$  do
  // Step 1 -- Initialize Active Zone
   $A_t \sim \text{Kaiming-uniform}$ 
  if  $t = 1$  then
     $B_t \leftarrow \mathbf{0}$  // standard LoRA cold start
  else
    Sample  $\tilde{B} \sim \mathcal{N}(\mathbf{0}, I)$ 
     $B_t \leftarrow \tilde{B} - Q_{\text{pool}}(Q_{\text{pool}}^\top \tilde{B})$ 
     $B_t \leftarrow \text{QR}(B_t)$  // init orthogonal to all pool bases
  end
  // Step 2 -- Initialize Mixing Matrix  $M_t$ 
   $r_{\text{tot}} \leftarrow (t-1) \cdot r$ 
   $U_t, V_t \in \mathbb{R}^{r_{\text{tot}} \times k}$ , zero-pad from  $M_{t-1}$  (warm start)
  // Step 3 -- Training Loop
  while not converged do
    Forward:  $\hat{y} = W_0 x + \Delta W_t(x)$ 
    Backward, then project gradient:
     $g \leftarrow \nabla_{B_t} \mathcal{L}$ 
    for  $j = 1$  to  $t-1$  do
       $g \leftarrow g - Q_j(Q_j^\top g)$  // Gram-Schmidt onto  $\text{span}(Q_j)^\perp$ 
    end
    Apply projected gradient  $g$  to  $B_t$ 
    Update  $A_t, U_t, V_t$  (no projection)
  end
  // Step 4 -- Commit to Basis Pool
   $[Q_t, R_t] \leftarrow \text{QR}(B_t)$  //  $Q_t^\top Q_t = I$ ; lossless factorisation
   $C_t \leftarrow R_t A_t$  //  $Q_t C_t x = B_t A_t x$  preserved exactly
  Freeze  $(Q_t, C_t) \rightarrow \text{Basis Pool}$ 
  Freeze and archive  $M_t$ 
  Reset:  $B_t \leftarrow \mathbf{0}$ ,
   $A_t \sim \text{Kaiming}$ 
end

```

Figure 2.2: TGR-LoRA Overall Algorithm

Chapter 3

System Design and Implementation

3.1 System Overview

Our project consists of four main components. First, the model and tokenizer are loaded from Hugging Face. Second, PEFT LoRA and TGR-LoRA is injected into selected linear modules (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj for Qwen-2.5-3B-Instruct). Next, the trainer processes the tasks sequentially, saving a checkpoint and performing an evaluation upon the completion of each task. Finally, the analysis tool saves the score matrix and generates plots.

3.2 Repository Structure

Our repository structure and related content is as follows.

File	Purpose
<code>main.py</code>	Loads configuration, model, tokenizer, and related trainer.
<code>run_experiments.sh</code>	Runs Sequential LoRA and selected TGR-LoRA configurations.
<code>summarise_results.py</code>	Reads run histories and prints summary metrics.
<code>sequential_lora.yaml</code>	Sequential LoRA configuration.
<code>ortho_hist_k4.yaml</code>	TGR-LoRA $mtk = 4$ configuration.
<code>ortho_hist_k8.yaml</code>	TGR-LoRA $mtk = 8$ configuration.
<code>ortho_hist_k16.yaml</code>	TGRTGR-LoRA $mtk = 16$ configuration.
<code>config.py</code>	Experiment Configuration and validation.
<code>data_trace.py</code>	TRACE dataset loading and collation.
<code>eval_trace.py</code>	Task-specific answer parsing and scoring.
<code>trainers.py</code>	TGR-LoRA and Sequential LoRA trainers.
<code>ortho_hist_lora.py</code>	TGR-LoRA layer and manager implementation.
<code>checkpointing.py</code>	Task-level checkpoint saving and loading.
<code>analysis_utils.py</code>	Score matrix and reuse heatmap generation.

Table 3.1: Main repository components.

3.3 Sequential LoRA Baseline Implementation

Sequential LoRA is implemented using the PEFT framework. The base Qwen model remains frozen, with only the LoRA A and B matrices being trainable during the training process. The same LoRA adapter is utilized across all tasks. This serves as the benchmark to assess whether Sequential LoRA suffers from catastrophic forgetting of earlier tasks.

The number of trainable parameters in this benchmark is approximately 29.9 million. This aligns with the parameter count obtained when applying a rank-16 LoRA to the attention projection and MLP projection layers of the Qwen2.5-3B-Instruct model, which validates that the base model freezing logic has been successfully applied.

3.4 TGR-LoRA Implementation

The TGR-LoRA wrapper replaces selected `nn.Linear` modules. Each wrapper stores:

- the frozen base linear layer;
- the active LoRA matrices `lora_A` and `lora_B`;
- a frozen `basis_pool` containing previous basis and coefficient blocks;

- the current task mixer matrices `mt_U` and `mt_V`;
- a backward hook that projects active `lora_B` gradients away from historical basis directions.

At the beginning of task 1, the current active branch is initialized the same as standard LoRA. In subsequent tasks, the active LoRA B is initialized orthogonally to the historical basis pool. Once the training of current task is finished, the current active update will be QR decomposed and store into the historical pool.

Chapter 4

Testing and Experimental Validation

4.1 Evaluation Method

The final implementation evaluates the model after the completion/training of each task. Each row of the score matrix corresponds to the model after the completion of training for a task, while each column corresponds to a task being evaluated. The training sequence for the final five tasks is as follows:

$$\text{C-STANCE} \rightarrow \text{FOMC} \rightarrow \text{ScienceQA} \rightarrow \text{NumGLUE-cm} \rightarrow \text{NumGLUE-ds}.$$

The task evaluation metrics are:

- C-STANCE: accuracy
- FOMC: accuracy
- ScienceQA: accuracy
- NumGLUE-cm: exact match
- NumGLUE-ds: exact match

The continual learning metrics are:

$$AA_t = \frac{1}{t} \sum_{i=1}^t S_{t,i},$$

and

$$\text{BWT}_t = \frac{1}{t-1} \sum_{i=1}^{t-1} (S_{t,i} - S_{i,i}),$$

where $S_{t,i}$ is the score on task i after training through task t .

We choose two classical continual learning metrics. AA_t represents the model’s average performance across all trained tasks after completing the training for the first t tasks. This metric reflects the model’s overall performance as it continuously acquires new knowledge.

BWT_t illustrates the impact of learning new tasks on previously learned tasks. By calculating the difference between a past task’s score at the current moment and its score after the first training, it measures whether the model has suffered from catastrophic forgetting.

4.2 Sequential LoRA Five-Task Baseline

Table 4.1 reports the performance of the Sequential LoRA baseline. The revised results demonstrate that Sequential LoRA serves as a robust baseline in this experiment. Upon completing training for the FOMC task, its C-STANCE score rose slightly from 0.6000 to 0.6100, yielding a corresponding BWT value of +0.0100. Following the subsequent completion of the ScienceQA and NumGLUE-cm tasks, the baseline’s performance remained stable. After finishing the NumGLUE-cm task, its AA score reached 0.7051, with a BWT value of +0.0300.

Upon completing the final task NumGLUE-ds, Sequential LoRA achieved a final AA score of 0.6393 and a final BWT value of -0.0234 , which shows a slight forgetting degree. However, we also note that this outcome is decided according to the experimental configuration: when we set the model’s maximum generation length to just 512 with 500 training samples, the degree of forgetting observed in Sequential LoRA was relatively low. However, when this limit was increased from 512 to 1024, and the number of training samples increased from 500 to 1000, the model exhibited more serious forgetting.

Table 4.1: Sequential LoRA Performance

Trained Up To	C-ST	FOMC	SciQ	Num-cm	Num-ds	AA $\uparrow$	BWT $\uparrow$	FWT $\uparrow$
C-STANCE	0.6000	–	–	–	–	0.6000	0.0000	0.0000
FOMC	0.6100	0.7300	–	–	–	0.6700	+0.0100	0.0000
ScienceQA	0.6000	0.7300	0.8200	–	–	0.7167	0.0000	0.0000
NumGLUE-cm	0.6200	0.7500	0.8700	0.5802	–	0.7051	+0.0300	0.0000
NumGLUE-ds	0.6100	0.7100	0.6500	0.6667	0.5600	0.6393	-0.0234	0.0000

4.3 TGR-LoRA Five-Task Comparison

For TGR-LoRA, we evaluated three settings for the historical mixing rank: $mt_k = 4$, $mt_k = 8$, and $mt_k = 16$. This mixing rank represents the size of the historical reuse operator for each task.

A larger mt_k gives the model greater degrees of freedom to recombine historical low-rank activations, while simultaneously introducing more trainable parameters into the reuse path.

The final results indicate that, in the five-task setting, $mt_k = 8$ achieved the best final AA and BWT scores, yielding a final AA of 0.6296 and a final BWT of -0.0400 . Furthermore, across various configurations, the model’s degree of forgetting consistently followed a pattern of firstly increasing and then decreasing. This trend also persisted after we increased the maximum generation length and the number of training samples. Moreover, under this configuration, the performance gap between TGR-LoRA and Sequential LoRA was smaller.

4.3.1 TGR-LoRA with $mt_k = 4$

Table 4.2 presents the performance of TGR-LoRA under the $mt_k = 4$ configuration. This configuration exhibited the most severe early forgetting phenomenon. Upon completion of the FOMC task, the C-STANCE score dropped from 0.6000 to 0.4400, corresponding to a BWT value of -0.1600 . Subsequent tasks led to a partial recovery of the model’s performance: After completing the ScienceQA task, the BWT improved to -0.0300 , while after completing the NumGLUE-cm task, the BWT briefly turned positive, reaching $+0.0100$. However, upon the completion of the final NumGLUE-ds task, the final AA value was 0.6182, while the final BWT declined to -0.0498 . Therefore, the $mt_k = 4$ configuration failed to provide a stable and effective mitigation of forgetting.

Table 4.2: TGR-LoRA ($mt_k = 4$) Performance

Trained Up To	C-ST	FOMC	SciQ	Num-cm	Num-ds	AA $\uparrow$	BWT $\uparrow$	FWT $\uparrow$
C-STANCE	0.6000	–	–	–	–	0.6000	0.0000	0.0000
FOMC	0.4400	0.7100	–	–	–	0.5750	-0.1600	0.0000
ScienceQA	0.5700	0.6800	0.8200	–	–	0.6900	-0.0300	0.0000
NumGLUE-cm	0.5900	0.7100	0.8600	0.5802	–	0.6851	+0.0100	0.0000
NumGLUE-ds	0.5700	0.7100	0.7000	0.5309	0.5800	0.6182	-0.0498	0.0000

4.3.2 TGR-LoRA with $mt_k = 8$

Table 4.3 presents the performance of TGR-LoRA under the $mt_k = 8$ configuration, which represents the strongest-performing TGR-LoRA configuration. Upon completion of the FOMC task, the C-STANCE score dropped from 0.6000 to 0.4900, yielding a corresponding BWT value of -0.1100 . This result still indicates the early forgetting problem. However, its performance in subsequent stages was more robust than that of the $mt_k = 4$ setting. After completing the ScienceQA task, the BWT improved to -0.0400 . Upon the completion of the NumGLUE-cm task, the BWT further improved to -0.0200 . And after the entire sequence of five tasks, the $mt_k = 8$ configuration finally achieved an AA score of 0.6296 and a BWT value of -0.0400 .

Table 4.3: TGR-LoRA ($mt_k = 8$) Performance

Trained Up To	C-ST	FOMC	SciQ	Num-cm	Num-ds	AA $\uparrow$	BWT $\uparrow$	FWT $\uparrow$
C-STANCE	0.6000	–	–	–	–	0.6000	0.0000	0.0000
FOMC	0.4900	0.7800	–	–	–	0.6350	-0.1100	0.0000
ScienceQA	0.5500	0.7500	0.7900	–	–	0.6967	-0.0400	0.0000
NumGLUE-cm	0.5800	0.7300	0.8000	0.5679	–	0.6695	-0.0200	0.0000
NumGLUE-ds	0.5600	0.7400	0.7100	0.5679	0.5700	0.6296	-0.0400	0.0000

4.3.3 TGR-LoRA with $mt_k = 16$

Table 4.4 reports the performance of TGR-LoRA with $mt_k = 16$. Among all tested configurations, this setting employs the largest historical mixing rank (the same as the LoRA rank 16). During the second-stage tasks, it exhibited less early forgetting compared to the $mt_k = 4$ and $mt_k = 8$ settings. Upon completion of the FOMC task, the C-STANCE metric dropped from 0.6000 to 0.5100, yielding a corresponding BWT value of -0.0900 . However, this advantage did not persist throughout the entire task sequence. After completing the ScienceQA task, the BWT shifted to -0.0750 . Upon finishing the NumGLUE-ds task, the final BWT changed to -0.0687 , which was the weakest final BWT score among all TGR-LoRA configurations. This suggests that simply increasing the mixing rank does not always guarantee improved reuse of historical information.

Table 4.4: TGR-LoRA ($mt_k = 16$) Performance

Trained Up To	C-ST	FOMC	SciQ	Num-cm	Num-ds	AA $\uparrow$	BWT $\uparrow$	FWT $\uparrow$
C-STANCE	0.6000	–	–	–	–	0.6000	0.0000	0.0000
FOMC	0.5100	0.7500	–	–	–	0.6300	-0.0900	0.0000
ScienceQA	0.5100	0.6900	0.8200	–	–	0.6733	-0.0750	0.0000
NumGLUE-cm	0.5500	0.7100	0.8000	0.5802	–	0.6601	-0.0367	0.0000
NumGLUE-ds	0.5600	0.7100	0.6500	0.5556	0.6000	0.6151	-0.0687	0.0000

4.3.4 Final Method Comparison

Table 4.5 shows a comparison of the final performance. Across all completed experiments, Sequential LoRA achieved the best final AA and BWT scores. Among the TGR-LoRA configurations, the configuration with $mt_k = 8$ performed the best, followed by $mt_k = 4$ and $mt_k = 16$.

Consequently, the final results are mixed. Although the TGR-LoRA successfully incorporates historical memory capabilities and trainable reuse of historical information, it currently fails to consistently outperform Sequential LoRA. Furthermore, likely due to the significant difference between the five selected datasets, the current model has only minimal reuse of historical task information at each layer (with values less than 0.001). We also observed that as the number of training samples increases, the performance gap between TGR-LoRA and Sequential LoRA narrows. Therefore, our method appears to be better suited for scenarios of high task similarity and larger training samples, as well as larger model generation length. Future directions should explore the integration of task-aware retrieval or task-specific inference mechanisms to ensure that, when evaluating prior tasks, the system utilizes the historical states most relevant to them.

Table 4.5: Final Comparison After the Five-Task Sequence

Method	C-ST	FOMC	SciQ	Num-cm	Num-ds	Final AA $\uparrow$	Final BWT $\uparrow$
Sequential LoRA	0.6100	0.7100	0.6500	0.6667	0.5600	0.6393	-0.0234
TGR-LoRA ($mt_k = 4$)	0.5700	0.7100	0.7000	0.5309	0.5800	0.6182	-0.0498
TGR-LoRA ($mt_k = 8$)	0.5600	0.7400	0.7100	0.5679	0.5700	0.6296	-0.0400
TGR-LoRA ($mt_k = 16$)	0.5600	0.7100	0.6500	0.5556	0.6000	0.6151	-0.0687

Chapter 5

Reproducibility, User Manual, and Bill of Materials

5.1 Code Repository

The final code repository is available at <https://github.com/by325-hub/ECE-5996-design-proj.git> or <https://github.com/byu959397-cell/ECE-5996-design-proj-code.git> if the previous one is not accessible.

The repository contains the Python source files, YAML configuration files, run scripts, README, and result artifact instructions.

5.2 Environment Setup

Listing 5.1: Basic environment setup.

```
1 python -m venv .venv
2 source .venv/bin/activate
3 pip install -r requirements.txt
```

For AutoDL or Colab, the following commands were useful:

Listing 5.2: Cloud environment notes.

```
1 export HF_ENDPOINT="https://hf-mirror.com"
2 pip uninstall -y torchao
3 export PYTORCH_CUDA_ALLOC_CONF=max_split_size_mb:128
```

5.3 Dataset Layout

Listing 5.3: Expected TRACE-style dataset layout.

```
1 TRACE/
2   C-STANCE/
3     train.json
4     eval.json
```

```

5     test.json
6   FOMC/
7     train.json
8     eval.json
9     test.json
10  ScienceQA/
11    train.json
12    eval.json
13    test.json
14  NumGLUE-cm/
15    train.json
16    eval.json
17    test.json
18  NumGLUE-ds/
19    train.json
20    eval.json
21    test.json

```

5.4 Running Experiments

Listing 5.4: Run Sequential LoRA only.

```
1 python main.py --config configs/sequential_lora.yaml
```

Listing 5.5: Run OrthoHist-LoRA k=4 only.

```
1 python main.py --config configs/ortho_hist_k4.yaml
```

Listing 5.6: Run Sequential LoRA and OrthoHist-LoRA through the shell script.

```
1 bash run_experiments.sh 0 4
```

Listing 5.7: Run only OrthoHist-LoRA and skip Sequential LoRA.

```
1 RUN_SEQ=0 bash run_experiments.sh 0 4
```

5.5 Expected Outputs

Each run writes to the output directory defined in its YAML file. Important files are:

- `history.json`: task-level scores, AA, BWT, and FWT.
- `score_matrix_raw.json`: raw score matrix.
- `analysis/score_matrix.png`: score matrix heatmap.
- `analysis/reuse_heatmap.png`: OrthoHist reuse heatmap.
- `logs/events.jsonl`: JSONL training and evaluation log.
- `checkpoints/task_XX/DONE`: task completion marker for resume.

5.6 Software Bill of Materials

Item	Purpose	Cost
Python	Main programming language	\$0
PyTorch	Tensor computation and model training	\$0
Transformers	Qwen model and tokenizer loading	\$0
PEFT	Sequential LoRA baseline	\$0
NumPy, SciPy	Analysis and numerical utilities	\$0
Matplotlib	Score matrix and reuse heatmap visualization	\$0
TRACE-style dataset	Continual learning validation	\$0
Cloud GPU compute	Colab / AutoDL training and debugging	Variable

Table 5.1: Software bill of materials.

Bibliography

- [1] E. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” 2021.
- [2] TRACE Benchmark. <https://github.com/BeyondrXX/TRACE.git>
- [3] Qwen Team. <https://huggingface.co/Qwen/Qwen2.5-3B-Instruct>
- [4] ChatGPT and Gemini were utilized to assist in polishing portions of the text, debugging and optimizing the code, and evaluating the TRACE evaluation logic. The core concepts and strategies of all the proposed solutions and ideas are the author’s.

Appendix A

Appendix A: Code Listings

A.1 Project Entry Point

Listing A.1: main.py: Project Entry Point

```
1  #!/usr/bin/env python3
2  import argparse
3  import pathlib
4
5  import torch
6  from transformers import AutoModelForCausalLM, AutoTokenizer
7
8  from src.config import load_config
9  from src.ortho_hist_lora import OrthoHistManager
10 from src.trainers import ContinualTrainer, SequentialLoRATrainer, set_seed
11
12
13 def build_model_and_tokenizer(cfg):
14     print(f"Loading {cfg.model_name}")
15
16     tok = AutoTokenizer.from_pretrained(
17             cfg.model_name,
18             use_fast=cfg.use_fast_tokenizer,
19             padding_side=cfg.padding_side,
20             trust_remote_code=True,
21     )
22     if tok.pad_token is None:
23             tok.pad_token = tok.eos_token
24
25     dtype = torch.bfloat16 if cfg.bf16 else torch.float32
26     model = AutoModelForCausalLM.from_pretrained(
27             cfg.model_name,
28             torch_dtype=dtype,
29             trust_remote_code=True,
30     )
31
32     # Training does not need KV cache. This also avoids warnings in some models.
33     if hasattr(model, "config"):
34             model.config.use_cache = False
35
36     return model, tok
37
38
```

```

39 def inject_peft_lora(model, cfg):
40     from peft import LoraConfig, TaskType, get_peft_model
41
42     lora_cfg = LoraConfig(
43         r=cfg.rank,
44         lora_alpha=cfg.alpha,
45         lora_dropout=cfg.dropout,
46         target_modules=cfg.target_modules,
47         task_type=TaskType.CAUSAL_LM,
48         bias="none",
49     )
50     return get_peft_model(model, lora_cfg)
51
52
53 def parse_args():
54     parser = argparse.ArgumentParser()
55     parser.add_argument("--config", required=True)
56     parser.add_argument("--output_dir", default=None)
57     parser.add_argument("--method", default=None)
58     parser.add_argument("--mt_rank", type=int, default=None)
59     parser.add_argument("--rank", type=int, default=None)
60     return parser.parse_args()
61
62
63 def apply_cli_overrides(cfg, args):
64     if args.output_dir is not None:
65         cfg.output_dir = args.output_dir
66     if args.method is not None:
67         cfg.method = args.method
68     if args.mt_rank is not None:
69         cfg.mt_rank = args.mt_rank
70     if args.rank is not None:
71         cfg.rank = args.rank
72     return cfg
73
74
75 def main():
76     args = parse_args()
77     cfg = apply_cli_overrides(load_config(args.config), args)
78
79     pathlib.Path(cfg.output_dir).mkdir(parents=True, exist_ok=True)
80
81     set_seed(cfg.seed)
82     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
83
84     print(f"device: {device}")
85     print(f"method: {cfg.method}")
86     print(f"output_dir: {cfg.output_dir}")
87     print(f"tasks: {', '.join(cfg.train_tasks)}")
88
89     model, tok = build_model_and_tokenizer(cfg)
90     model.to(device)
91
92     if cfg.method == "ortho_hist":
93         manager = OrthoHistManager(
94             model,
95             target_modules=cfg.target_modules,
96             rank=cfg.rank,
97             alpha=cfg.alpha,
98             dropout=cfg.dropout,
99             mt_rank=cfg.mt_rank,

```

```

100         device=device,
101     )
102     manager.inject()
103     ContinualTrainer(cfg, tok, model, manager, device).train_all_tasks()
104
105     elif cfg.method == "sequential_lora":
106         model = inject_peft_lora(model, cfg)
107         model.to(device)
108         SequentialLoRATrainer(cfg, tok, model, device).train_all_tasks()
109
110     else:
111         raise ValueError(f"Unknown method: {cfg.method}")
112
113     print(f"\nDone. Results in {cfg.output_dir}")
114
115
116 if __name__ == "__main__":
117     main()

```

A.2 Experiment Runner

Listing A.2: run_experiments.sh: Experiment Runner

```

1  #!/bin/bash
2  # run_experiments.sh
3  # Run Sequential LoRA and OrthoHist-LoRA on the five TRACE tasks.
4  # Usage:
5  #   bash run_experiments.sh [gpu_id] [k ...]
6  # Examples:
7  #   bash run_experiments.sh 0          # run k=4,8,16 if configs exist
8  #   bash run_experiments.sh 0 8        # run only k=8
9  #   bash run_experiments.sh 0 4 8 16
10
11  set -euo pipefail
12
13  GPU=${1:-0}
14  shift || true
15
16  KS=("$@")
17  if [ ${#KS[@]} -eq 0 ]; then
18      KS=(4 8 16)
19  fi
20
21  export CUDA_VISIBLE_DEVICES="$GPU"
22  CONFIG_DIR=${CONFIG_DIR:-configs}
23
24  RUN_SEQ=${RUN_SEQ:-1}
25  RUN_ORTHO=${RUN_ORTHO:-1}
26
27  run_config() {
28      local label="$1"
29      local cfg="$2"
30
31      if [ ! -f "$cfg" ]; then
32          echo "[Skip] $label: config not found: $cfg"
33          return 0
34      fi
35

```

```

36     echo ""
37     echo ">>> $label"
38     echo "    config: $cfg"
39     python main.py --config "$cfg"
40 }
41
42
43 echo "==== Continual LoRA Experiments ====="
44 echo "GPU: $GPU"
45 echo "CONFIG_DIR: $CONFIG_DIR"
46 echo "OrthoHist k values: ${KS[*]}"
47 echo ""
48
49 if [ "$RUN_SEQ" = "1" ]; then
50     run_config "Sequential LoRA" "$CONFIG_DIR/sequential_lora.yaml"
51 else
52     echo "[Skip] Sequential LoRA disabled by RUN_SEQ=0"
53 fi
54
55 if [ "$RUN_ORTHO" = "1" ]; then
56     for K in "${KS[@]}; do
57         run_config "OrthoHist-LoRA mt_rank=$K" "$CONFIG_DIR/ortho_hist_k${K}.yaml"
58     done
59 else
60     echo "[Skip] OrthoHist-LoRA disabled by RUN_ORTHO=0"
61 fi
62
63
64 echo ""
65 echo "==== Finished ====="
66 echo "To generate the table:"
67 echo "    python summarise_results.py"

```

A.3 Result Summary Script

Listing A.3: summarise_results.py: Result Summary Script

```

1  #!/usr/bin/env python3
2  from __future__ import annotations
3
4  import json
5  from pathlib import Path
6  from typing import Any, Dict, Optional
7
8  TASKS = [
9      "C-STANCE",
10     "FOMC",
11     "ScienceQA",
12     "NumGLUE-cm",
13     "NumGLUE-ds",
14 ]
15
16 SHORT = ["C-ST", "FOMC", "SciQ", "Num-cm", "Num-ds"]
17
18 RUNS = {
19     "Sequential LoRA": "outputs/sequential_lora",
20     "OrthoHist k=4": "outputs/ortho_hist_k4",
21     "OrthoHist k=8": "outputs/ortho_hist_k8",

```

```

22     "OrthoHist k=16": "outputs/ortho_hist_k16",
23 }
24
25
26 def load_json(path):
27     if not path.exists():
28         return None
29     with open(path, "r", encoding="utf-8") as f:
30         return json.load(f)
31
32
33 def load_history(run_dir: str):
34     return load_json(Path(run_dir) / "history.json")
35
36
37 def final_row_from_history(history):
38     if not history:
39         return {
40             "scores": {},
41             "aa": None,
42             "bwt": None,
43             "fwt": None,
44         }
45
46     last = history[-1]
47
48     # New trainer format.
49     scores = last.get("seen_scores")
50     aa = last.get("average_accuracy")
51
52     # Backward compatibility with older SequentialLoRATrainer history.
53     if scores is None:
54         scores = last.get("scores", {})
55     if aa is None:
56         aa = last.get("aa")
57
58     return {
59         "scores": scores or {},
60         "aa": aa,
61         "bwt": last.get("bwt"),
62         "fwt": last.get("fwt"),
63     }
64
65
66 def fmt(v):
67     return "" if v is None else f"{v * 100:.1f}"
68
69
70 def build_rows():
71     rows = []
72     for name, run_dir in RUNS.items():
73         history = load_history(run_dir)
74         row = {"name": name, "run_dir": run_dir}
75         row.update(final_row_from_history(history))
76         rows.append(row)
77     return rows
78
79
80 def print_console_table(rows):
81     header = (
82         f"{'Method':<22}"

```

```

83     + """.join(f"{s:>8}" for s in SHORT)
84     + f"{'AA':>8}{'BWT':>8}{'FWT':>8}"
85 )
86 print(header)
87 print("-" * len(header))
88
89 for row in rows:
90     line = f"{row['name']:<22}"
91     for task in TASKS:
92         line += f"{fmt(row['scores'].get(task)):>8}"
93     line += f"{fmt(row['aa']):>8}"
94     line += f"{fmt(row['bwt']):>8}"
95     line += f"{fmt(row['fwf']):>8}"
96     print(line)
97
98 missing = [r for r in rows if r["aa"] is None]
99 if missing:
100     print("\nMissing runs:")
101     for r in missing:
102         print(f" - {r['name']}: {r['run_dir']}/history.json not found")
103
104 print("\nValues are percentages. BWT < 0 indicates forgetting.")
105
106
107 def print_latex_table(rows):
108     print(r"\begin{table}[t]")
109     print(r"\centering")
110     print(r"\small")
111     cols = "l" + "c" * (len(TASKS) + 3)
112     print(r"\begin{tabular}{%s}" % cols)
113     print(r"\toprule")
114     print("Method & " + " & ".join(SHORT) + r" & AA & BWT & FWT \\")
115     print(r"\midrule")
116
117     for row in rows:
118         cells = [row["name"]]
119         for task in TASKS:
120             cells.append(fmt(row["scores"].get(task)))
121         cells.append(fmt(row["aa"]))
122         cells.append(fmt(row["bwt"]))
123         cells.append(fmt(row["fwf"]))
124         print(" & ".join(cells) + r" \\")
125
126     print(r"\bottomrule")
127     print(r"\end{tabular}")
128     print(
129         r"\caption{Continual learning results on five TRACE tasks "
130         r"(Qwen2.5-3B-Instruct, LoRA rank=16). "
131         r"AA denotes average accuracy over seen tasks after the full task sequence. "
132         r"BWT$<$0 indicates forgetting.}"
133     )
134     print(r"\label{tab:main_results}")
135     print(r"\end{table}")
136
137
138 def main():
139     rows = build_rows()
140     print_console_table(rows)
141     print()
142     print_latex_table(rows)
143

```



```
144
145 if __name__ == "__main__":
146     main()
```

A.4 Python Requirements

Listing A.4: requirements.txt: Python Requirements

```
1 torch>=2.1.0
2 transformers>=4.40.0
3 peft>=0.10.0
4 accelerate>=0.27.0
5
6 safetensors>=0.4.2
7 huggingface_hub>=0.23.0
8 tokenizers>=0.19.0
9 packaging>=23.0
10
11 numpy>=1.24.0
12 pyyaml>=6.0
13 tqdm>=4.65.0
14 matplotlib>=3.7.0
15 scipy>=1.10.0
16 datasets>=2.18.0
17 rouge-score>=0.1.2
```

A.5 Configuration: Sequential LoRA

Listing A.5: configs/sequential_lora.yaml: Configuration: Sequential LoRA

```
1 seed: 42
2 output_dir: ./outputs/sequential_lora
3 trace_root: ./TRACE
4 model_name: Qwen/Qwen2.5-3B-Instruct
5 use_fast_tokenizer: false
6 padding_side: left
7 max_length: 1024
8 per_device_batch_size: 2
9 grad_accum_steps: 4
10 num_epochs: 3
11 learning_rate: 0.0002
12 weight_decay: 0.0
13 warmup_ratio: 0.03
14 max_grad_norm: 1.0
15 bf16: true
16 compile_model: false
17 log_every: 10
18 eval_every_epoch: false
19 save_every_task: true
20 method: sequential_lora
21 rank: 16
22 alpha: 32
23 dropout: 0.0
24 debug_eval: false
25 target_modules:
```

```

26 - q_proj
27 - k_proj
28 - v_proj
29 - o_proj
30 - gate_proj
31 - up_proj
32 - down_proj
33 train_tasks:
34 - C-STANCE
35 - FOMC
36 - ScienceQA
37 - NumGLUE-cm
38 - NumGLUE-ds
39 eval_tasks:
40 - C-STANCE
41 - FOMC
42 - ScienceQA
43 - NumGLUE-cm
44 - NumGLUE-ds

```

A.6 Configuration: OrthoHist-LoRA $k=4$

Listing A.6: configs/ortho_hist_k4.yaml: Configuration: OrthoHist-LoRA $k=4$

```

1 seed: 42
2 output_dir: ./outputs/ortho_hist_k4
3 trace_root: ./TRACE
4 model_name: Qwen/Qwen2.5-3B-Instruct
5 use_fast_tokenizer: false
6 padding_side: left
7 max_length: 1024
8
9 per_device_batch_size: 2
10 grad_accum_steps: 4
11 num_epochs: 3
12 learning_rate: 2.0e-4
13 weight_decay: 0.0
14 warmup_ratio: 0.03
15 max_grad_norm: 1.0
16 bf16: true
17 compile_model: false
18
19 log_every: 10
20 eval_every_epoch: false
21 save_every_task: true
22 debug_eval: false
23
24 method: ortho_hist
25
26 rank: 16
27 alpha: 32
28 dropout: 0.0
29 mt_rank: 4
30
31 target_modules:
32 - q_proj
33 - k_proj
34 - v_proj

```

```

35 - o_proj
36 - gate_proj
37 - up_proj
38 - down_proj
39
40 train_tasks:
41 - C-STANCE
42 - FOMC
43 - ScienceQA
44 - NumGLUE-cm
45 - NumGLUE-ds
46
47 eval_tasks:
48 - C-STANCE
49 - FOMC
50 - ScienceQA
51 - NumGLUE-cm
52 - NumGLUE-ds

```

A.7 Configuration: OrthoHist-LoRA k=8

Listing A.7: configs/ortho_hist_k8.yaml: Configuration: OrthoHist-LoRA k=8

```

1 seed: 42
2 output_dir: ./outputs/ortho_hist_k8
3 trace_root: ./TRACE
4 model_name: Qwen/Qwen2.5-3B-Instruct
5 use_fast_tokenizer: false
6 padding_side: left
7 max_length: 1024
8 per_device_batch_size: 2
9 grad_accum_steps: 4
10 num_epochs: 3
11 learning_rate: 0.0002
12 weight_decay: 0.0
13 warmup_ratio: 0.03
14 max_grad_norm: 1.0
15 bf16: true
16 compile_model: false
17 log_every: 10
18 eval_every_epoch: false
19 save_every_task: true
20 method: ortho_hist
21 rank: 16
22 alpha: 32
23 dropout: 0.0
24 mt_rank: 8
25 debug_eval: false
26 target_modules:
27 - q_proj
28 - k_proj
29 - v_proj
30 - o_proj
31 - gate_proj
32 - up_proj
33 - down_proj
34 train_tasks:
35 - C-STANCE

```

```

36 - FOMC
37 - ScienceQA
38 - NumGLUE-cm
39 - NumGLUE-ds
40 eval_tasks:
41 - C-STANCE
42 - FOMC
43 - ScienceQA
44 - NumGLUE-cm
45 - NumGLUE-ds

```

A.8 Configuration: OrthoHist-LoRA k=16

Listing A.8: configs/ortho_hist_k16.yaml: Configuration: OrthoHist-LoRA k=16

```

1 seed: 42
2 output_dir: ./outputs/ortho_hist_k16
3 trace_root: ./TRACE
4 model_name: Qwen/Qwen2.5-3B-Instruct
5 use_fast_tokenizer: false
6 padding_side: left
7 max_length: 1024
8
9 per_device_batch_size: 2
10 grad_accum_steps: 4
11 num_epochs: 3
12 learning_rate: 2.0e-4
13 weight_decay: 0.0
14 warmup_ratio: 0.03
15 max_grad_norm: 1.0
16 bf16: true
17 compile_model: false
18
19 log_every: 10
20 eval_every_epoch: false
21 save_every_task: true
22 debug_eval: false
23
24 method: ortho_hist
25
26 rank: 16
27 alpha: 32
28 dropout: 0.0
29 mt_rank: 16
30
31 target_modules:
32 - q_proj
33 - k_proj
34 - v_proj
35 - o_proj
36 - gate_proj
37 - up_proj
38 - down_proj
39
40 train_tasks:
41 - C-STANCE
42 - FOMC
43 - ScienceQA

```

```

44     - NumGLUE-cm
45     - NumGLUE-ds
46
47     eval_tasks:
48     - C-STANCE
49     - FOMC
50     - ScienceQA
51     - NumGLUE-cm
52     - NumGLUE-ds

```

A.9 Configuration Loader

Listing A.9: src/config.py: Configuration Loader

```

1  from dataclasses import dataclass, field
2  from pathlib import Path
3  from typing import List
4
5  import yaml
6
7  # !
8  @dataclass
9  class TrainConfig:
10     # paths and model
11     seed: int = 42
12     output_dir: str = "./outputs/run"
13     trace_root: str = "./TRACE"
14     model_name: str = "Qwen/Qwen2.5-3B-Instruct"
15     use_fast_tokenizer: bool = False
16     padding_side: str = "left"
17     max_length: int = 1024
18
19     # training
20     per_device_batch_size: int = 2
21     grad_accum_steps: int = 4
22     num_epochs: int = 3
23     learning_rate: float = 2.0e-4
24     weight_decay: float = 0.0
25     warmup_ratio: float = 0.03
26     max_grad_norm: float = 1.0
27     bf16: bool = True
28     compile_model: bool = False
29
30     # logging and checkpointing
31     log_every: int = 10
32     eval_every_epoch: bool = False
33     save_every_task: bool = True
34     debug_eval: bool = False
35
36     # method: "ortho_hist" | "sequential_lora"
37     method: str = "ortho_hist"
38
39     # LoRA
40     rank: int = 16
41     alpha: int = 32
42     dropout: float = 0.0
43     target_modules: List[str] = field(default_factory=lambda: [
44         "q_proj", "k_proj", "v_proj", "o_proj",

```

```

45     "gate_proj", "up_proj", "down_proj",
46 ]))
47
48 # OrthoHist-specific
49 mt_rank: int = 8
50
51 # task schedule
52 train_tasks: List[str] = field(default_factory=lambda: [
53     "C-STANCE", "FOMC", "ScienceQA", "NumGLUE-cm", "NumGLUE-ds",
54 ])
55 eval_tasks: List[str] = field(default_factory=lambda: [
56     "C-STANCE", "FOMC", "ScienceQA", "NumGLUE-cm", "NumGLUE-ds",
57 ])
58
59
60 def load_config(path):
61     with open(path, "r", encoding="utf-8") as f:
62         raw = yaml.safe_load(f) or {}
63
64     valid_keys = set(TrainConfig.__dataclass_fields__.keys())
65     unknown = sorted(set(raw.keys()) - valid_keys)
66     if unknown:
67         raise ValueError(f"Unknown config keys in {path}: {unknown}")
68
69     cfg = TrainConfig(**raw)
70     Path(cfg.output_dir).mkdir(parents=True, exist_ok=True)
71     return cfg

```

A.10 TRACE Data Loader

Listing A.10: src/data_trace.py: TRACE Data Loader

```

1  from __future__ import annotations
2
3  import json
4  from dataclasses import dataclass
5  from pathlib import Path
6
7  import torch
8  from torch.utils.data import DataLoader, Dataset
9
10
11  TRACE_TASKS = [
12      "C-STANCE",
13      "FOMC",
14      "ScienceQA",
15      "NumGLUE-cm",
16      "NumGLUE-ds",
17  ]
18
19  TASK_METRICS = {
20      "C-STANCE": "accuracy",
21      "FOMC": "accuracy",
22      "ScienceQA": "accuracy",
23      "NumGLUE-cm": "exact_match",
24      "NumGLUE-ds": "exact_match",
25  }
26

```

```

27
28 @dataclass
29 class EncodedExample:
30     input_ids: torch.Tensor
31     labels: torch.Tensor
32     prompt_text: str
33     answer_text: str
34     task_name: str
35
36
37 class TraceSFTDataset(Dataset):
38     def __init__(
39         self,
40         path,
41         tokenizer,
42         max_length=1024,
43         task_name=None,
44         split="train",
45         max_eval_samples=100,
46     ):
47         path = Path(path)
48         if not path.exists():
49             raise FileNotFoundError(f"TRACE split file not found: {path}")
50
51         with open(path, "r", encoding="utf-8") as f:
52             self.data = json.load(f)
53
54         self.split = split
55         self.tokenizer = tokenizer
56         self.max_length = int(max_length)
57         self.task_name = task_name or path.parent.name
58
59         if split in {"eval", "test"} and max_eval_samples is not None:
60             self.data = self.data[: int(max_eval_samples)]
61
62     def __len__(self):
63         return len(self.data)
64
65     def _build_prompt_text(self, prompt):
66         return self.tokenizer.apply_chat_template(
67             [{"role": "user", "content": prompt}],
68             tokenize=False,
69             add_generation_prompt=True,
70         )
71
72     def _build_full_text(self, prompt, answer):
73         return self.tokenizer.apply_chat_template(
74             [
75                 {"role": "user", "content": prompt},
76                 {"role": "assistant", "content": answer},
77             ],
78             tokenize=False,
79             add_generation_prompt=False,
80         )
81
82     def _encode_with_mask(self, prompt, answer):
83         prompt_text = self._build_prompt_text(prompt)
84         full_text = self._build_full_text(prompt, answer)
85
86         prompt_ids = self.tokenizer(prompt_text, add_special_tokens=False).input_ids
87         full_ids = self.tokenizer(full_text, add_special_tokens=False).input_ids

```

```

88     answer_ids = full_ids[len(prompt_ids):]
89
90     if len(full_ids) <= self.max_length:
91         kept_prompt_ids = prompt_ids
92         kept_answer_ids = answer_ids
93     else:
94         min_answer_tokens = max(1, min(len(answer_ids), 128))
95         prompt_budget = max(0, self.max_length - min_answer_tokens)
96
97         kept_prompt_ids = prompt_ids[-prompt_budget:] if prompt_budget > 0 else []
98         answer_budget = self.max_length - len(kept_prompt_ids)
99         kept_answer_ids = answer_ids[:answer_budget]
100
101         if kept_answer_ids and len(answer_ids) > len(kept_answer_ids):
102             eos_ids = self.tokenizer(
103                 self.tokenizer.eos_token or "<|im_end|>",
104                 add_special_tokens=False,
105             ).input_ids
106             if eos_ids:
107                 kept_answer_ids[-1] = eos_ids[-1]
108
109     kept_ids = kept_prompt_ids + kept_answer_ids
110     labels = [-100] * len(kept_prompt_ids) + kept_answer_ids.copy()
111
112     return EncodedExample(
113         input_ids=torch.tensor(kept_ids, dtype=torch.long),
114         labels=torch.tensor(labels, dtype=torch.long),
115         prompt_text=prompt,
116         answer_text=answer,
117         task_name=self.task_name,
118     )
119
120     def __getitem__(self, idx):
121         sample = self.data[idx]
122
123         if "prompt" not in sample or "answer" not in sample:
124             raise KeyError(
125                 f"Each TRACE sample must contain 'prompt' and 'answer'. "
126                 f"Bad sample in {self.task_name}/{self.split}: {sample}"
127             )
128
129         if self.split in {"eval", "test"}:
130             return {
131                 "prompt_text": sample["prompt"],
132                 "answer_text": sample["answer"],
133                 "task_name": self.task_name,
134             }
135
136         encoded = self._encode_with_mask(sample["prompt"], sample["answer"])
137         return {
138             "input_ids": encoded.input_ids,
139             "labels": encoded.labels,
140             "prompt_text": encoded.prompt_text,
141             "answer_text": encoded.answer_text,
142             "task_name": encoded.task_name,
143         }
144
145     def left_pad_stack(seqs, pad_value):
146         max_len = max(x.size(0) for x in seqs)
147         out = []

```



```

149     for x in seqs:
150         pad_len = max_len - x.size(0)
151         if pad_len > 0:
152             pad = torch.full((pad_len,), pad_value, dtype=x.dtype)
153             out.append(torch.cat([pad, x], dim=0))
154         else:
155             out.append(x)
156     return torch.stack(out, dim=0)
157
158
159 def make_trace_collator(tokenizer):
160     pad_token_id = tokenizer.pad_token_id
161
162     def collate(features):
163         if "input_ids" not in features[0]:
164             return {
165                 "prompt_text": [f["prompt_text"] for f in features],
166                 "answer_text": [f["answer_text"] for f in features],
167                 "task_name": [f["task_name"] for f in features],
168             }
169
170         input_ids = left_pad_stack([f["input_ids"] for f in features], pad_token_id)
171         labels = left_pad_stack([f["labels"] for f in features], -100)
172         attention_mask = (input_ids != pad_token_id).long()
173
174         return {
175             "input_ids": input_ids,
176             "labels": labels,
177             "attention_mask": attention_mask,
178             "prompt_text": [f["prompt_text"] for f in features],
179             "answer_text": [f["answer_text"] for f in features],
180             "task_name": [f["task_name"] for f in features],
181         }
182
183     return collate
184
185
186 def build_trace_loader(
187     trace_root,
188     task_name,
189     split,
190     tokenizer,
191     batch_size,
192     max_length,
193     shuffle=False,
194     num_workers=0,
195 ):
196     path = Path(trace_root) / task_name / f"{split}.json"
197
198     dataset = TraceSFTDataset(
199         path,
200         tokenizer=tokenizer,
201         max_length=max_length,
202         task_name=task_name,
203         split=split,
204     )
205
206     return DataLoader(
207         dataset,
208         batch_size=batch_size,
209         shuffle=shuffle,

```

```

210         num_workers=num_workers,
211         collate_fn=make_trace_collator(tokenizer),
212         pin_memory=torch.cuda.is_available(),
213     )
214
215
216 def build_task_loaders(
217     trace_root,
218     tasks,
219     tokenizer,
220     batch_size,
221     max_length,
222 ):
223     loaders = {}
224
225     for task in tasks:
226         loaders[task] = {
227             "train": build_trace_loader(
228                 trace_root,
229                 task,
230                 "train",
231                 tokenizer,
232                 batch_size,
233                 max_length,
234                 shuffle=True,
235             ),
236             "eval": build_trace_loader(
237                 trace_root,
238                 task,
239                 "eval",
240                 tokenizer,
241                 20,
242                 max_length,
243                 shuffle=False,
244             ),
245             "test": build_trace_loader(
246                 trace_root,
247                 task,
248                 "test",
249                 tokenizer,
250                 20,
251                 max_length,
252                 shuffle=False,
253             ),
254         }
255
256     return loaders

```

A.11 TRACE Evaluation Utilities

Listing A.11: src/eval_trace.py: TRACE Evaluation Utilities

```

1 from __future__ import annotations
2
3 import re
4 from dataclasses import dataclass
5 from decimal import Decimal, InvalidOperation
6

```

```

7 from .data_trace import TASK_METRICS
8
9 def normalize_text(text):
10     if text is None:
11         return ""
12     return " ".join(str(text).strip().split()).lower()
13
14
15 def parse_explicit_choice(text, valid_choices=("A", "B", "C")):
16     if text is None:
17         return None
18
19     raw = str(text).strip()
20     if not raw:
21         return None
22
23     choices = " ".join(valid_choices)
24     choices_lower = choices.lower()
25
26     m = re.search(
27         rf"(?:the\s+correct\s+answer|correct\s+answer|the\s+answer|answer|choice|option|label|result|stance)"
28         rf"\s*(?:is|==|:|)?\s*"
29         rf"([{choices}-{choices_lower}])"
30         rf"(?:\s|$\s|[\.\.\,;])",
31         raw,
32         flags=re.IGNORECASE,
33     )
34     if m:
35         return m.group(1).upper()
36
37     m = re.match(
38         rf"^\s*[\(\[\]?\s*([{choices}])\s*[\)\]"]?"
39         rf"(?:\s|$\s|[\.\.\,;])",
40         raw,
41     )
42     if m:
43         return m.group(1).upper()
44
45     m = re.match(
46         rf"^\s*[\(\[\]?\s*([{choices_lower}])\s*[\)\]"]?"
47         rf"(?:\s|$\s|[\.\.\,;])",
48         raw,
49     )
50     if m:
51         return m.group(1).upper()
52
53     m = re.search(
54         rf"(?:choose|select|selected|chooses|choosing|option|choice)\s*"
55         rf"([{choices}-{choices_lower}])"
56         rf"(?:\s|$\s|[\.\.\,;])",
57         raw,
58         flags=re.IGNORECASE,
59     )
60     if m:
61         return m.group(1).upper()
62
63     return None
64
65
66 def parse_cstance_answer(text):

```

```

67     label = parse_explicit_choice(text, valid_choices=("A", "B", "C"))
68     if label is not None:
69         return label
70
71     if text is None:
72         return None
73
74     t = normalize_text(text)
75
76     m = re.search(
77         r"(?:||answer|label|stance)"
78         r"\s*(?:||=|:|)?\s*"
79         r"(|)",
80         t,
81         flags=re.IGNORECASE,
82     )
83     if m:
84         return {"": "A", "": "B", "": "C"}[m.group(1)]
85
86     m = re.match(r"^s*(|)(?:s|$|,|\.|!)", t)
87     if m:
88         return {"": "A", "": "B", "": "C"}[m.group(1)]
89
90     found = set()
91
92     if re.search(r"(|)", t):
93         found.add("B")
94
95     if re.search(r"(?!|)", t):
96         found.add("A")
97
98     if re.search(r"(|)", t):
99         found.add("C")
100
101     if len(found) == 1:
102         return next(iter(found))
103
104     return None
105
106
107 def parse_fomc_answer(text):
108     label = parse_explicit_choice(text, valid_choices=("A", "B", "C"))
109     if label is not None:
110         return label
111
112     if text is None:
113         return None
114
115     t = normalize_text(text)
116     found = set()
117
118     if re.search(r"\bdovish\b", t, flags=re.IGNORECASE):
119         found.add("A")
120
121     if re.search(r"\bhawkish\b", t, flags=re.IGNORECASE):
122         found.add("B")
123
124     if re.search(r"\bneutral\b", t, flags=re.IGNORECASE):
125         found.add("C")
126
127     if len(found) == 1:

```

```

128         return next(iter(found))
129
130     return None
131
132
133 def parse_scienceqa_answer(text):
134     return parse_explicit_choice(
135         text,
136         valid_choices=("A", "B", "C", "D", "E"),
137     )
138
139
140 def parse_answer_for_task(task_name, text):
141     if task_name == "C-STANCE":
142         return parse_cstance_answer(text)
143
144     if task_name == "FOMC":
145         return parse_fomc_answer(text)
146
147     if task_name == "ScienceQA":
148         return parse_scienceqa_answer(text)
149
150     return normalize_text(text)
151
152
153 MONTHS = {
154     "january", "february", "march", "april", "may", "june",
155     "july", "august", "september", "october", "november", "december",
156 }
157
158
159 def parse_decimal(text):
160     if text is None:
161         return None
162
163     s = str(text).strip().replace(",", "")
164
165     try:
166         return Decimal(s)
167     except InvalidOperation:
168         return None
169
170
171 def extract_explicit_number(text):
172     if text is None:
173         return None
174
175     raw = str(text).replace(",", "")
176
177     m = re.search(
178         r"(?:the\s+answer|answer|final\s+answer|result|)"
179         r"\s*(?:is|==|=|:|)?\s*"
180         r"([-+]?\d+(?:\.\d+)?)",
181         raw,
182         flags=re.IGNORECASE,
183     )
184
185     if m:
186         return parse_decimal(m.group(1))
187
188     return None

```

```

189
190
191 def extract_leading_number(text):
192     if text is None:
193         return None
194
195     raw = str(text).strip().replace(",", "")
196
197     m = re.match(
198         r"^\s*([-+]?\d+(?:\.\d+)?) (?:\s|[$|[%\.\,\;\:\)\]])",
199         raw,
200     )
201
202     if m:
203         return parse_decimal(m.group(1))
204
205     return None
206
207
208 def extract_last_number(text):
209     if text is None:
210         return None
211
212     raw = str(text).replace(",", "")
213     nums = re.findall(r"[-+]?\d+(?:\.\d+)?", raw)
214
215     if not nums:
216         return None
217
218     return parse_decimal(nums[-1])
219
220
221 def extract_number_answer(text):
222     num = extract_explicit_number(text)
223     if num is not None:
224         return num
225
226     num = extract_leading_number(text)
227     if num is not None:
228         return num
229
230     return extract_last_number(text)
231
232
233 def month_pattern():
234     return (
235         r"(?:january|february|march|april|may|june|july|august|"
236         r"september|october|november|december)"
237     )
238
239
240 def extract_explicit_month(text):
241     if text is None:
242         return None
243
244     raw = str(text).strip()
245
246     m = re.search(
247         rf"(?:the\s+answer|answer|final\s+answer|result|)"
248         rf"\s*(?:is|||=|:|)?\s*"
249         rf"({month_pattern()})"

```

```

250         rf"(?:\s|${[\.\.\}\],:;])",
251         raw,
252         flags=re.IGNORECASE,
253     )
254
255     if m:
256         return normalize_text(m.group(1))
257
258     return None
259
260
261 def extract_leading_month(text):
262     if text is None:
263         return None
264
265     raw = str(text).strip()
266
267     m = re.match(
268         rf"^\s*({month_pattern()}) (?:\s|${[\.\.\}\],:;])",
269         raw,
270         flags=re.IGNORECASE,
271     )
272
273     if m:
274         return normalize_text(m.group(1))
275
276     return None
277
278
279 def extract_last_month(text):
280     if text is None:
281         return None
282
283     t = normalize_text(text)
284     hits = []
285
286     for month in MONTHS:
287         for m in re.finditer(rf"\b{month}\b", t):
288             hits.append((m.start(), month))
289
290     if not hits:
291         return None
292
293     hits.sort(key=lambda x: x[0])
294     return hits[-1][1]
295
296
297 def extract_month_answer(text):
298     month = extract_explicit_month(text)
299     if month is not None:
300         return month
301
302     month = extract_leading_month(text)
303     if month is not None:
304         return month
305
306     return extract_last_month(text)
307
308
309 def clean_final_text_answer(text):
310     if text is None:

```

```

311         return ""
312
313     raw = str(text).strip()
314
315     m = re.search(
316         r"(?:the\s+answer|answer|final\s+answer|result|)"
317         r"\s*(?:is|==|:|)?\s*"
318         r"([a-zA-Z]+)",
319         raw,
320         flags=re.IGNORECASE,
321     )
322     if m:
323         return normalize_text(m.group(1)).strip(".,;:!?")
324
325     return normalize_text(raw).strip(".,;:!?")
326
327
328 def exact_match_score(preds, refs):
329     correct = 0
330     total = len(preds)
331
332     for pred, ref in zip(preds, refs):
333         ref_num = parse_decimal(ref)
334
335         if ref_num is not None:
336             pred_num = extract_number_answer(pred)
337
338             if pred_num is not None and pred_num == ref_num:
339                 correct += 1
340
341         else:
342             ref_month = extract_month_answer(ref)
343
344             if ref_month is not None:
345                 pred_month = extract_month_answer(pred)
346
347                 if pred_month is not None and pred_month == ref_month:
348                     correct += 1
349
350             else:
351                 pred_clean = clean_final_text_answer(pred)
352                 ref_clean = clean_final_text_answer(ref)
353
354                 if pred_clean == ref_clean:
355                     correct += 1
356
357     return correct / max(1, total)
358
359
360 def accuracy_score(preds, refs, task_name):
361     correct = 0
362     total = len(preds)
363
364     for pred, ref in zip(preds, refs):
365         if task_name in {"C-STANCE", "FOMC", "ScienceQA"}:
366             parsed_pred = parse_answer_for_task(task_name, pred)
367             parsed_ref = parse_answer_for_task(task_name, ref)
368
369             if (
370                 parsed_pred is not None
371                 and parsed_ref is not None

```



```

372         and parsed_pred == parsed_ref
373     ):
374         correct += 1
375
376     else:
377         pred_clean = normalize_text(pred)
378         ref_clean = normalize_text(ref)
379
380         if pred_clean == ref_clean:
381             correct += 1
382
383     return correct / max(1, total)
384
385
386 @dataclass
387 class TaskEvalResult:
388     task_name: str
389     metric_name: str
390     metric_value: float
391     num_examples: int
392
393
394 def score_task(task_name, preds, refs, sources=None):
395     metric_name = TASK_METRICS.get(task_name, "exact_match")
396
397     if metric_name == "accuracy":
398         value = accuracy_score(preds, refs, task_name)
399
400     elif metric_name == "exact_match":
401         value = exact_match_score(preds, refs)
402
403     else:
404         raise ValueError(f"Unsupported metric: {metric_name}")
405
406     return TaskEvalResult(
407         task_name=task_name,
408         metric_name=metric_name,
409         metric_value=value,
410         num_examples=len(preds),
411     )
412
413
414 def parse_for_debug(task_name, pred, ref):
415     metric_name = TASK_METRICS.get(task_name, "exact_match")
416
417     if metric_name == "accuracy":
418         parsed_pred = parse_answer_for_task(task_name, pred)
419         parsed_ref = parse_answer_for_task(task_name, ref)
420         correct = (
421             parsed_pred is not None
422             and parsed_ref is not None
423             and parsed_pred == parsed_ref
424         )
425         return parsed_pred, parsed_ref, correct
426
427     if metric_name == "exact_match":
428         ref_num = parse_decimal(ref)
429
430         if ref_num is not None:
431             parsed_pred = extract_number_answer(pred)
432             parsed_ref = ref_num

```

```

433         correct = (
434             parsed_pred is not None
435             and parsed_ref is not None
436             and parsed_pred == parsed_ref
437         )
438         return parsed_pred, parsed_ref, correct
439
440     ref_month = extract_month_answer(ref)
441
442     if ref_month is not None:
443         parsed_pred = extract_month_answer(pred)
444         parsed_ref = ref_month
445         correct = (
446             parsed_pred is not None
447             and parsed_ref is not None
448             and parsed_pred == parsed_ref
449         )
450         return parsed_pred, parsed_ref, correct
451
452     parsed_pred = clean_final_text_answer(pred)
453     parsed_ref = clean_final_text_answer(ref)
454     correct = (
455         parsed_pred is not None
456         and parsed_ref is not None
457         and parsed_pred == parsed_ref
458     )
459     return parsed_pred, parsed_ref, correct
460
461     return None, None, False
462
463
464 def debug_parse_examples(task_name, preds, refs, n=20):
465     for i, (pred, ref) in enumerate(zip(preds[:n], refs[:n])):
466         parsed_pred, parsed_ref, correct = parse_for_debug(task_name, pred, ref)
467
468         print("=" * 80)
469         print(f"idx: {i}")
470         print(f"task: {task_name}")
471         print(f"raw pred: {repr(pred)}")
472         print(f"raw ref : {repr(ref)}")
473         print(f"parsed pred: {parsed_pred}")
474         print(f"parsed ref : {parsed_ref}")
475         print(f"correct: {correct}")
476
477
478 def build_debug_rows(task_name, preds, refs, prompts):
479     rows = []
480
481     for prompt, pred, ref in zip(prompts, preds, refs):
482         parsed_pred, parsed_ref, correct = parse_for_debug(task_name, pred, ref)
483
484         rows.append({
485             "task": task_name,
486             "prompt": prompt,
487             "ref": ref,
488             "pred": pred,
489             "parsed_ref": str(parsed_ref) if parsed_ref is not None else None,
490             "parsed_pred": str(parsed_pred) if parsed_pred is not None else None,
491             "correct": correct,
492         })
493

```

```

494     return rows
495
496
497 def average_accuracy(results):
498     return sum(results.values()) / len(results) if results else 0.0
499
500
501 def backward_transfer(score_matrix, trained_tasks):
502     if len(trained_tasks) <= 1:
503         return 0.0
504
505     current = trained_tasks[-1]
506     current_row = score_matrix.get(current, {})
507
508     diffs = []
509     for task in trained_tasks[:-1]:
510         if task in current_row and task in score_matrix and task in score_matrix[task]:
511             diffs.append(current_row[task] - score_matrix[task][task])
512
513     return sum(diffs) / len(diffs) if diffs else 0.0
514
515
516 def forward_transfer(zero_shot_scores, score_matrix, trained_tasks):
517     # If zero-shot scores are not explicitly computed, FWT is undefined in this
518     # experiment code path. Return 0.0 to keep the existing table format stable.
519     if len(trained_tasks) <= 1 or not zero_shot_scores:
520         return 0.0
521
522     total, count = 0.0, 0
523
524     for i, task in enumerate(trained_tasks[1:], 1):
525         prev = trained_tasks[i - 1]
526         prev_row = score_matrix.get(prev, {})
527
528         if task in prev_row and task in zero_shot_scores:
529             total += prev_row[task] - zero_shot_scores[task]
530             count += 1
531
532     return total / count if count else 0.0

```

A.12 Training Loops

Listing A.12: src/trainers.py: Training Loops

```

1  import json
2  import math
3  import random
4  from pathlib import Path
5
6  import numpy as np
7  import torch
8  from torch.optim import AdamW
9  from tqdm import tqdm
10 from transformers import get_linear_schedule_with_warmup
11
12 from .analysis_utils import collect_reuse_matrix, save_reuse_heatmap, save_score_matrix
13 from .checkpointing import load_checkpoint, save_checkpoint
14 from .data_trace import build_task_loaders

```

```

15 from .eval_trace import (
16     average_accuracy,
17     backward_transfer,
18     build_debug_rows,
19     debug_parse_examples,
20     forward_transfer,
21     score_task,
22 )
23
24
25 def set_seed(seed):
26     random.seed(seed)
27     np.random.seed(seed)
28     torch.manual_seed(seed)
29     torch.cuda.manual_seed_all(seed)
30
31
32 def _make_optimizer_and_scheduler(params, cfg, n_steps):
33     opt = AdamW(params, lr=cfg.learning_rate, weight_decay=cfg.weight_decay)
34     warmup = int(cfg.warmup_ratio * n_steps)
35     sch = get_linear_schedule_with_warmup(opt, warmup, n_steps)
36     return opt, sch
37
38
39 def _find_last_done_task(output_dir, n_tasks):
40     """Return the last consecutively completed task id."""
41     last_done = 0
42     ckpt_root = Path(output_dir) / "checkpoints"
43
44     for tid in range(1, n_tasks + 1):
45         ckpt_dir = ckpt_root / f"task_{tid:02d}"
46         done_flag = ckpt_dir / "DONE"
47         state_file = ckpt_dir / "model_state.pt"
48
49         if done_flag.exists() and state_file.exists():
50             last_done = tid
51         else:
52             break
53
54     return last_done
55
56
57 class JsonLogger:
58     def __init__(self, output_dir):
59         p = Path(output_dir) / "logs"
60         p.mkdir(parents=True, exist_ok=True)
61         self.path = p / "events.jsonl"
62         self.path.touch(exist_ok=True)
63
64     def log(self, payload):
65         self.path.parent.mkdir(parents=True, exist_ok=True)
66         with open(self.path, "a", encoding="utf-8") as f:
67             f.write(json.dumps(payload, ensure_ascii=False) + "\n")
68
69
70 class ContinualTrainer:
71     def __init__(self, cfg, tokenizer, model, manager, device):
72         self.cfg = cfg
73         self.tok = tokenizer
74         self.model = model
75         self.manager = manager

```

```

76     self.device = device
77     self.logger = JsonLogger(cfg.output_dir)
78
79     self.loaders = build_task_loaders(
80         trace_root=cfg.trace_root,
81         tasks=cfg.train_tasks,
82         tokenizer=tokenizer,
83         batch_size=cfg.per_device_batch_size,
84         max_length=cfg.max_length,
85     )
86
87     self.score_matrix = {}
88     self.zero_shot_scores = {}
89     self.reuse_snapshots = []
90     self.history = []
91
92     def _save_history(self):
93         out = Path(self.cfg.output_dir)
94         out.mkdir(parents=True, exist_ok=True)
95         with open(out / "history.json", "w", encoding="utf-8") as f:
96             json.dump(self.history, f, indent=2, ensure_ascii=False)
97
98     def _save_score_matrix(self):
99         save_score_matrix(
100             self.score_matrix,
101             self.cfg.train_tasks,
102             self.cfg.output_dir,
103         )
104
105         out = Path(self.cfg.output_dir)
106         out.mkdir(parents=True, exist_ok=True)
107         with open(out / "score_matrix_raw.json", "w", encoding="utf-8") as f:
108             json.dump(self.score_matrix, f, indent=2, ensure_ascii=False)
109
110     def _save_reuse_snapshots(self):
111         out = Path(self.cfg.output_dir)
112         out.mkdir(parents=True, exist_ok=True)
113         with open(out / "reuse_snapshots.json", "w", encoding="utf-8") as f:
114             json.dump(self.reuse_snapshots, f, indent=2, ensure_ascii=False)
115
116     def _restore_history_and_scores(self):
117         out = Path(self.cfg.output_dir)
118
119         hp = out / "history.json"
120         if hp.exists():
121             with open(hp, encoding="utf-8") as f:
122                 self.history = json.load(f)
123
124         smp = out / "score_matrix_raw.json"
125         if smp.exists():
126             with open(smp, encoding="utf-8") as f:
127                 self.score_matrix = json.load(f)
128             print(f"[Resume] restored score_matrix from {smp}")
129         else:
130             print("[Resume WARNING] score_matrix_raw.json not found; rebuilding from history.")
131             self.score_matrix = {}
132             for entry in self.history:
133                 key = entry.get("task_name") or entry.get("task")
134                 if key is not None:
135                     self.score_matrix[key] = dict(entry.get("seen_scores", {}))
136

```

```

137 def _restore_reuse_snapshots(self):
138     snap_path = Path(self.cfg.output_dir) / "reuse_snapshots.json"
139     if snap_path.exists():
140         with open(snap_path, encoding="utf-8") as f:
141             self.reuse_snapshots = json.load(f)
142             print(f"[Resume] restored {len(self.reuse_snapshots)} reuse snapshots")
143
144 def _preallocate_orthohist_pool_for_resume(self, n_committed_tasks):
145     """Create basis_pool slots before loading an OrthoHist checkpoint."""
146     if n_committed_tasks <= 0:
147         return
148
149     for layer in self.manager.layers.values():
150         layer.basis_pool = torch.nn.ParameterList()
151         device = layer.lora_B.device
152         dtype = layer.lora_B.dtype
153
154         for _ in range(n_committed_tasks):
155             basis = torch.empty(
156                 layer.out_features,
157                 layer.rank,
158                 device=device,
159                 dtype=dtype,
160             )
161             coeff = torch.empty(
162                 layer.rank,
163                 layer.in_features,
164                 device=device,
165                 dtype=dtype,
166             )
167             layer.basis_pool.append(torch.nn.Parameter(basis, requires_grad=False))
168             layer.basis_pool.append(torch.nn.Parameter(coeff, requires_grad=False))
169
170         layer._n_pool = n_committed_tasks
171
172 def _mark_done(self, task_id):
173     done_flag = (
174         Path(self.cfg.output_dir)
175         / "checkpoints"
176         / f"task_{task_id:02d}"
177         / "DONE"
178     )
179     done_flag.parent.mkdir(parents=True, exist_ok=True)
180     done_flag.touch()
181
182 def _train_one_epoch(self, task_name, optimizer, scheduler, epoch):
183     self.model.train()
184     loader = self.loaders[task_name]["train"]
185     total_loss = 0.0
186     batch_cnt = 0
187     optimizer.zero_grad(set_to_none=True)
188
189     pbar = tqdm(loader, desc=f"[{task_name}] epoch {epoch}", leave=False)
190     for step, batch in enumerate(pbar, 1):
191         batch = {
192             k: v.to(self.device) if isinstance(v, torch.Tensor) else v
193             for k, v in batch.items()
194         }
195
196         out = self.model(
197             input_ids=batch["input_ids"],

```

```

198         attention_mask=batch["attention_mask"],
199         labels=batch["labels"],
200     )
201     loss = out.loss / self.cfg.grad_accum_steps
202     loss.backward()
203
204     if step % self.cfg.grad_accum_steps == 0:
205         torch.nn.utils.clip_grad_norm_(
206             self.manager.trainable_parameters(), self.cfg.max_grad_norm
207         )
208         optimizer.step()
209         scheduler.step()
210         optimizer.zero_grad(set_to_none=True)
211
212         total_loss += float(out.loss.item())
213         batch_cnt += 1
214         if batch_cnt % self.cfg.log_every == 0:
215             pbar.set_postfix(loss=f"{total_loss / batch_cnt:.4f}")
216
217     if len(loader) % self.cfg.grad_accum_steps != 0:
218         torch.nn.utils.clip_grad_norm_(
219             self.manager.trainable_parameters(), self.cfg.max_grad_norm
220         )
221         optimizer.step()
222         scheduler.step()
223         optimizer.zero_grad(set_to_none=True)
224
225     return total_loss / max(1, batch_cnt)
226
227 @torch.no_grad()
228 def _generate(self, task_name, split="test"):
229     self.model.eval()
230     loader = self.loaders[task_name][split]
231     preds, refs, prompts = [], [], []
232
233     for batch in tqdm(loader, desc=f"eval {task_name}/{split}", leave=False):
234         prompt_texts = [
235             self.tok.apply_chat_template(
236                 [{"role": "user", "content": p}],
237                 tokenize=False,
238                 add_generation_prompt=True,
239             )
240             for p in batch["prompt_text"]
241         ]
242
243         enc = self.tok(
244             prompt_texts,
245             return_tensors="pt",
246             padding=True,
247             truncation=True,
248             max_length=self.cfg.max_length,
249             add_special_tokens=False,
250         ).to(self.device)
251
252         if task_name in {"C-STANCE", "FOMC"}:
253             max_new_tokens = 8
254         elif task_name == "ScienceQA":
255             max_new_tokens = 64
256         else:
257             max_new_tokens = 128

```

```

259         out = self.model.generate(
260             input_ids=enc["input_ids"],
261             attention_mask=enc["attention_mask"],
262             max_new_tokens=max_new_tokens,
263             eos_token_id=self.tok.eos_token_id,
264             pad_token_id=self.tok.pad_token_id,
265             do_sample=False,
266         )
267
268         input_len = enc["input_ids"].shape[1]
269         for i in range(out.size(0)):
270             pred = self.tok.decode(out[i][input_len:], skip_special_tokens=True)
271             preds.append(pred.strip())
272
273         refs.extend(batch["answer_text"])
274         prompts.extend(batch["prompt_text"])
275
276         if getattr(self.cfg, "debug_eval", False):
277             debug_parse_examples(task_name, preds, refs, n=10)
278
279         return {"preds": preds, "refs": refs, "prompts": prompts}
280
281     def _save_raw_predictions(self, row_key, task_name, split, gen_results):
282         debug_dir = Path(self.cfg.output_dir) / "debug_predictions"
283         debug_dir.mkdir(parents=True, exist_ok=True)
284
285         safe_row = str(row_key).replace("/", "_")
286         safe_task = str(task_name).replace("/", "_")
287         out_path = debug_dir / f"{safe_row}_{safe_task}_{split}.jsonl"
288
289         rows = build_debug_rows(
290             task_name=task_name,
291             preds=gen_results["preds"],
292             refs=gen_results["refs"],
293             prompts=gen_results["prompts"],
294         )
295
296         with open(out_path, "w", encoding="utf-8") as f:
297             for row in rows:
298                 f.write(json.dumps(row, ensure_ascii=False) + "\n")
299
300     def _eval_tasks(self, task_list, row_key, split="test"):
301         self.score_matrix.setdefault(row_key, {})
302         task_scores = {}
303
304         for task in task_list:
305             gen_results = self._generate(task, split=split)
306             self._save_raw_predictions(
307                 row_key=row_key,
308                 task_name=task,
309                 split=split,
310                 gen_results=gen_results,
311             )
312
313             res = score_task(
314                 task,
315                 gen_results["preds"],
316                 gen_results["refs"],
317                 gen_results["prompts"],
318             )
319

```



```

320         task_scores[task] = res.metric_value
321         self.score_matrix[row_key][task] = res.metric_value
322
323         self.logger.log({
324             "event": "task_eval",
325             "row": row_key,
326             "task": task,
327             "split": split,
328             "metric": res.metric_name,
329             "value": res.metric_value,
330             "num_examples": res.num_examples,
331         })
332
333     return task_scores
334
335 def train_one_task(self, task_name, task_id, seen):
336     self.manager.prepare_for_task(task_id)
337
338     n_steps = max(
339         1,
340         math.ceil(
341             len(self.loaders[task_name]["train"]) * self.cfg.num_epochs
342             / self.cfg.grad_accum_steps
343         ),
344     )
345     opt, sch = _make_optimizer_and_scheduler(
346         self.manager.trainable_parameters(), self.cfg, n_steps
347     )
348
349     self.logger.log({
350         "event": "task_start",
351         "task": task_name,
352         "task_id": task_id,
353         "trainable_params": sum(p.numel() for p in self.manager.trainable_parameters()),
354     })
355
356     for epoch in range(1, self.cfg.num_epochs + 1):
357         loss = self._train_one_epoch(task_name, opt, sch, epoch)
358         self.logger.log({
359             "event": "epoch_end",
360             "task": task_name,
361             "epoch": epoch,
362             "loss": loss,
363         })
364
365     print(f"[{task_name}] training done, running eval...")
366     self._eval_tasks(self.cfg.train_tasks, row_key=task_name, split="test")
367
368     if hasattr(self.manager, "collect_reuse_analysis"):
369         snapshot = self.manager.collect_reuse_analysis(task_name)
370         self.reuse_snapshots.append(snapshot)
371         self._save_reuse_snapshots()
372
373     self.manager.commit_task()
374
375     scores = {t: self.score_matrix[task_name][t] for t in seen}
376     aa = average_accuracy(scores)
377     bwt = backward_transfer(self.score_matrix, seen)
378     fwt = forward_transfer(self.zero_shot_scores, self.score_matrix, seen)
379
380     self.history.append({

```

```

381         "task_id": task_id,
382         "task_name": task_name,
383         "seen_scores": scores,
384         "average_accuracy": aa,
385         "bwt": bwt,
386         "fwt": fwt,
387     })
388
389     self._save_history()
390     self._save_score_matrix()
391
392     self.logger.log({
393         "event": "task_end",
394         "task": task_name,
395         "aa": aa,
396         "bwt": bwt,
397         "fwt": fwt,
398     })
399     print(f"[{task_name}] AA={aa:.4f} BWT={bwt:.4f} FWT={fwt:.4f}")
400
401     if self.cfg.save_every_task:
402         save_checkpoint(
403             self.cfg.output_dir,
404             task_id,
405             self.model,
406             extra={
407                 "task": task_name,
408                 "aa": aa,
409                 "bwt": bwt,
410                 "fwt": fwt,
411             },
412         )
413         self._mark_done(task_id)
414
415     def train_all_tasks(self):
416         seen = []
417         last_done = _find_last_done_task(self.cfg.output_dir, len(self.cfg.train_tasks))
418
419         if last_done > 0:
420             print(
421                 f"[Resume] Loading checkpoint from task {last_done} "
422                 f"({self.cfg.train_tasks[last_done - 1]})"
423             )
424             self._preallocate_orthohist_pool_for_resume(last_done)
425             load_checkpoint(self.cfg.output_dir, last_done, self.model)
426
427             for layer in self.manager.layers.values():
428                 layer._n_pool = last_done
429
430             self._restore_history_and_scores()
431             self._restore_reuse_snapshots()
432
433             for task_id, task in enumerate(self.cfg.train_tasks, 1):
434                 seen.append(task)
435                 if task_id <= last_done:
436                     print(f"[Resume] skipping {task} (already done)")
437                     continue
438                 self.train_one_task(task, task_id, seen)
439
440     self._finalise()
441     return {"history": self.history, "score_matrix": self.score_matrix}

```

```

442
443     def _finalise(self):
444         self._save_history()
445         self._save_score_matrix()
446
447         if self.reuse_snapshots:
448             matrix = collect_reuse_matrix(self.reuse_snapshots, self.cfg.train_tasks)
449             save_reuse_heatmap(matrix, self.cfg.train_tasks, self.cfg.output_dir)
450
451
452     class SequentialLoRATrainer:
453         def __init__(self, cfg, tokenizer, model, device):
454             self.cfg = cfg
455             self.tok = tokenizer
456             self.model = model
457             self.device = device
458             self.loaders = build_task_loaders(
459                 trace_root=cfg.trace_root,
460                 tasks=cfg.train_tasks,
461                 tokenizer=tokenizer,
462                 batch_size=cfg.per_device_batch_size,
463                 max_length=cfg.max_length,
464             )
465             self.score_matrix = {}
466             self.zero_shot_scores = {}
467             self.history = []
468             self.logger = JsonLogger(cfg.output_dir)
469
470             self._set_lora_trainable_only()
471
472         def _set_lora_trainable_only(self):
473             for _, p in self.model.named_parameters():
474                 p.requires_grad_(False)
475
476             trainable = []
477             for name, p in self.model.named_parameters():
478                 if "lora_A" in name or "lora_B" in name:
479                     p.requires_grad_(True)
480                     trainable.append(p)
481
482             if not trainable:
483                 raise RuntimeError("No LoRA parameters found in SequentialLoRATrainer.")
484
485         def _trainable_parameters(self):
486             return [p for p in self.model.parameters() if p.requires_grad]
487
488         def _save_history(self):
489             out = Path(self.cfg.output_dir)
490             out.mkdir(parents=True, exist_ok=True)
491             with open(out / "history.json", "w", encoding="utf-8") as f:
492                 json.dump(self.history, f, indent=2, ensure_ascii=False)
493
494         def _save_score_matrix(self):
495             save_score_matrix(
496                 self.score_matrix,
497                 self.cfg.train_tasks,
498                 self.cfg.output_dir,
499             )
500
501             out = Path(self.cfg.output_dir)
502             out.mkdir(parents=True, exist_ok=True)

```

```

503     with open(out / "score_matrix_raw.json", "w", encoding="utf-8") as f:
504         json.dump(self.score_matrix, f, indent=2, ensure_ascii=False)
505
506     def _restore_history_and_scores(self):
507         out = Path(self.cfg.output_dir)
508
509         hp = out / "history.json"
510         if hp.exists():
511             with open(hp, encoding="utf-8") as f:
512                 self.history = json.load(f)
513
514         smp = out / "score_matrix_raw.json"
515         if smp.exists():
516             with open(smp, encoding="utf-8") as f:
517                 self.score_matrix = json.load(f)
518             print(f"[Resume] restored score_matrix from {smp}")
519         else:
520             print("[Resume WARNING] score_matrix_raw.json not found; rebuilding from history.")
521             self.score_matrix = {}
522             for entry in self.history:
523                 key = entry.get("task_name") or entry.get("task")
524                 if key is not None:
525                     self.score_matrix[key] = dict(entry.get("seen_scores", {}))
526
527     def _mark_done(self, task_id):
528         done_flag = (
529             Path(self.cfg.output_dir)
530             / "checkpoints"
531             / f"task_{task_id:02d}"
532             / "DONE"
533         )
534         done_flag.parent.mkdir(parents=True, exist_ok=True)
535         done_flag.touch()
536
537     @torch.no_grad()
538     def _generate(self, task_name, split="test"):
539         self.model.eval()
540         loader = self.loaders[task_name][split]
541         preds, refs, prompts = [], [], []
542
543         for batch in tqdm(loader, desc=f"eval {task_name}/{split}", leave=False):
544             prompt_texts = [
545                 self.tok.apply_chat_template(
546                     [{"role": "user", "content": p}],
547                     tokenize=False,
548                     add_generation_prompt=True,
549                 )
550                 for p in batch["prompt_text"]
551             ]
552
553             enc = self.tok(
554                 prompt_texts,
555                 return_tensors="pt",
556                 padding=True,
557                 truncation=True,
558                 max_length=self.cfg.max_length,
559                 add_special_tokens=False,
560             ).to(self.device)
561
562             if task_name in {"C-STANCE", "FOMC"}:
563                 max_new_tokens = 8

```

```

564         elif task_name == "ScienceQA":
565             max_new_tokens = 64
566         else:
567             max_new_tokens = 128
568
569         out = self.model.generate(
570             input_ids=enc["input_ids"],
571             attention_mask=enc["attention_mask"],
572             max_new_tokens=max_new_tokens,
573             eos_token_id=self.tok.eos_token_id,
574             pad_token_id=self.tok.pad_token_id,
575             do_sample=False,
576         )
577
578         input_len = enc["input_ids"].shape[1]
579         for i in range(out.size(0)):
580             pred = self.tok.decode(out[i][input_len:], skip_special_tokens=True)
581             preds.append(pred.strip())
582
583             refs.extend(batch["answer_text"])
584             prompts.extend(batch["prompt_text"])
585
586         if getattr(self.cfg, "debug_eval", False):
587             debug_parse_examples(task_name, preds, refs, n=10)
588
589         return {"preds": preds, "refs": refs, "prompts": prompts}
590
591     def _save_raw_predictions(self, row_key, task_name, split, gen_results):
592         debug_dir = Path(self.cfg.output_dir) / "debug_predictions"
593         debug_dir.mkdir(parents=True, exist_ok=True)
594
595         safe_row = str(row_key).replace("/", "_")
596         safe_task = str(task_name).replace("/", "_")
597         out_path = debug_dir / f"{safe_row}_{safe_task}_{split}.jsonl"
598
599         rows = build_debug_rows(
600             task_name=task_name,
601             preds=gen_results["preds"],
602             refs=gen_results["refs"],
603             prompts=gen_results["prompts"],
604         )
605
606         with open(out_path, "w", encoding="utf-8") as f:
607             for row in rows:
608                 f.write(json.dumps(row, ensure_ascii=False) + "\n")
609
610     def _eval_tasks(self, task_list, row_key, split="test"):
611         self.score_matrix.setdefault(row_key, {})
612         task_scores = {}
613
614         for task in task_list:
615             gen_results = self._generate(task, split=split)
616             self._save_raw_predictions(
617                 row_key=row_key,
618                 task_name=task,
619                 split=split,
620                 gen_results=gen_results,
621             )
622
623             res = score_task(
624                 task,

```

```

625         gen_results["preds"],
626         gen_results["refs"],
627         gen_results["prompts"],
628     )
629
630     task_scores[task] = res.metric_value
631     self.score_matrix[row_key][task] = res.metric_value
632
633     self.logger.log({
634         "event": "task_eval",
635         "row": row_key,
636         "task": task,
637         "split": split,
638         "metric": res.metric_name,
639         "value": res.metric_value,
640         "num_examples": res.num_examples,
641     })
642
643     return task_scores
644
645 def _train_one_task(self, task_name, task_id):
646     self._set_lora_trainable_only()
647     trainable = self._trainable_parameters()
648     loader = self.loaders[task_name]["train"]
649
650     n_steps = max(
651         1,
652         math.ceil(
653             len(loader) * self.cfg.num_epochs
654             / self.cfg.grad_accum_steps
655         ),
656     )
657     opt, sch = _make_optimizer_and_scheduler(trainable, self.cfg, n_steps)
658     self.model.train()
659     opt.zero_grad(set_to_none=True)
660
661     self.logger.log({
662         "event": "task_start",
663         "task": task_name,
664         "task_id": task_id,
665         "trainable_params": sum(p.numel() for p in trainable),
666     })
667
668     for epoch in range(1, self.cfg.num_epochs + 1):
669         total_loss = 0.0
670         batch_cnt = 0
671         pbar = tqdm(loader, desc=f"seq_lora [{task_name}] ep{epoch}", leave=False)
672
673         for step, batch in enumerate(pbar, 1):
674             batch = {
675                 k: v.to(self.device) if isinstance(v, torch.Tensor) else v
676                 for k, v in batch.items()
677             }
678             out = self.model(
679                 input_ids=batch["input_ids"],
680                 attention_mask=batch["attention_mask"],
681                 labels=batch["labels"],
682             )
683             (out.loss / self.cfg.grad_accum_steps).backward()
684
685             if step % self.cfg.grad_accum_steps == 0:

```

```

686         torch.nn.utils.clip_grad_norm_(trainable, self.cfg.max_grad_norm)
687         opt.step()
688         sch.step()
689         opt.zero_grad(set_to_none=True)
690
691         total_loss += float(out.loss.item())
692         batch_cnt += 1
693         if batch_cnt % self.cfg.log_every == 0:
694             pbar.set_postfix(loss=f"{total_loss / batch_cnt:.4f}")
695
696     if len(loader) % self.cfg.grad_accum_steps != 0:
697         torch.nn.utils.clip_grad_norm_(trainable, self.cfg.max_grad_norm)
698         opt.step()
699         sch.step()
700         opt.zero_grad(set_to_none=True)
701
702     self.logger.log({
703         "event": "epoch_end",
704         "task": task_name,
705         "epoch": epoch,
706         "loss": total_loss / max(1, batch_cnt),
707     })
708
709     def train_all_tasks(self):
710         seen = []
711         last_done = _find_last_done_task(self.cfg.output_dir, len(self.cfg.train_tasks))
712
713         if last_done > 0:
714             print(
715                 f"[Resume] Loading Sequential LoRA checkpoint from task {last_done} "
716                 f"({self.cfg.train_tasks[last_done - 1]})"
717             )
718             load_checkpoint(self.cfg.output_dir, last_done, self.model)
719             self.model.to(self.device)
720             self._set_lora_trainable_only()
721             self._restore_history_and_scores()
722
723         for task_id, task in enumerate(self.cfg.train_tasks, 1):
724             seen.append(task)
725             if task_id <= last_done:
726                 print(f"[Resume] skipping {task} (already done)")
727                 continue
728
729             self._train_one_task(task, task_id)
730             self._eval_tasks(self.cfg.train_tasks, row_key=task, split="test")
731
732             scores = {t: self.score_matrix[task][t] for t in seen}
733             aa = average_accuracy(scores)
734             bwt = backward_transfer(self.score_matrix, seen)
735             fwt = forward_transfer(self.zero_shot_scores, self.score_matrix, seen)
736
737             self.history.append({
738                 "task_id": task_id,
739                 "task_name": task,
740                 "seen_scores": scores,
741                 "average_accuracy": aa,
742                 "bwt": bwt,
743                 "fwt": fwt,
744             })
745
746             self._save_history()

```

```

747         self._save_score_matrix()
748
749         self.logger.log({
750             "event": "task_end",
751             "task": task,
752             "aa": aa,
753             "bwt": bwt,
754             "fwt": fwt,
755         })
756         print(f"[seq_lora {task}] AA={aa:.4f} BWT={bwt:.4f} FWT={fwt:.4f}")
757
758         if self.cfg.save_every_task:
759             save_checkpoint(
760                 self.cfg.output_dir,
761                 task_id,
762                 self.model,
763                 extra={
764                     "task": task,
765                     "aa": aa,
766                     "bwt": bwt,
767                     "fwt": fwt,
768                 },
769             )
770             self._mark_done(task_id)
771
772         self._finalise()
773         return {"history": self.history, "score_matrix": self.score_matrix}
774
775     def _finalise(self):
776         self._save_history()
777         self._save_score_matrix()

```

A.13 OrthoHist-LoRA Implementation

Listing A.13: src/ortho_hist_lora.py: OrthoHist-LoRA Implementation

```

1  import math
2  from typing import Dict, List, Optional, Tuple
3  import torch
4  import torch.nn as nn
5  import torch.nn.functional as F
6
7  class OrthoHistLoRALinear(nn.Module):
8      def __init__(self, base_layer: nn.Linear, rank=16, alpha=32, dropout=0.0, mt_rank=8):
9          super().__init__()
10         self.base_layer = base_layer
11         self.rank = rank
12         self.alpha = alpha
13         self.scaling = alpha / rank
14         self.mt_rank = mt_rank
15         self.in_features = base_layer.in_features
16         self.out_features = base_layer.out_features
17
18         for p in self.base_layer.parameters():
19             p.requires_grad_(False)
20
21         self.dropout = nn.Dropout(dropout) if dropout > 0.0 else nn.Identity()
22

```



```

23     self.lora_A = nn.Parameter(torch.empty(rank, self.in_features))
24     self.lora_B = nn.Parameter(torch.empty(self.out_features, rank))
25     self._init_active_fresh()
26
27     # pool stores alternating (basis [d_out, r], coeff [r, d_in]) as frozen params
28     self.basis_pool = nn.ParameterList()
29     self._n_pool = 0
30
31     # current task's mixing matrix M_t = U_mt @ V_mt.T
32     self.mt_U: Optional[nn.Parameter] = None
33     self.mt_V: Optional[nn.Parameter] = None
34
35     # keep archived M_t's for analysis
36     # Store the Mt for each task (does not count as a parameter, the true parameters consist solely of
the basis pool, the active zone, and the current Mt, not including all Mts)
37     self._frozen_mt: List[Tuple[Optional[torch.Tensor], Optional[torch.Tensor]]] = []
38
39     self._hook = self.register_full_backward_hook(self._ortho_grad_hook)
40
41     def _init_active_fresh(self):
42         # standard LoRA init for task 1 kaiming: suitable for deep neural networks
43         nn.init.kaiming_uniform_(self.lora_A, a=math.sqrt(5))
44         nn.init.zeros_(self.lora_B)
45
46     def init_active_projected(self):
47         if self._n_pool == 0:
48             nn.init.kaiming_uniform_(self.lora_A, a=math.sqrt(5))
49             nn.init.zeros_(self.lora_B)
50             return
51
52         device = self.lora_B.device
53         dtype = self.lora_B.dtype
54         pool_B = self._cat_pool_basis().float()
55         # proj0pool
56         # QR
57         B_rand = torch.randn(self.out_features, self.rank, device=device).float()
58         # QR
59         B_proj = B_rand - pool_B @ (pool_B.T @ B_rand)
60         # mode reduced
61         q, _ = torch.linalg.qr(B_proj, mode="reduced")
62
63         with torch.no_grad():
64             self.lora_B.copy_(q.to(dtype=dtype, device=device))
65
66         nn.init.zeros_(self.lora_A)
67
68
69     def init_mt(self):
70         r_total = self._n_pool * self.rank
71
72         if r_total == 0:
73             self.mt_U = None
74             self.mt_V = None
75             return
76
77         device = self.lora_B.device
78         dtype = self.lora_B.dtype
79
80         U_new = torch.empty(
81             r_total,
82             self.mt_rank,

```

```

83         device=device,
84         dtype=dtype,
85     )
86
87     V_new = torch.empty(
88         r_total,
89         self.mt_rank,
90         device=device,
91         dtype=dtype,
92     )
93     # AB
94     nn.init.normal_(U_new, mean=0.0, std=0.001)
95     nn.init.normal_(V_new, mean=0.0, std=0.001)
96
97     self.mt_U = nn.Parameter(U_new)
98     self.mt_V = nn.Parameter(V_new)
99
100     # check
101     def forward(self, x):
102         base = self.base_layer(x)
103         x_d = self.dropout(x)
104
105         active = F.linear(F.linear(x_d, self.lora_A), self.lora_B)
106
107         pool_out = x.new_zeros(active.shape)
108         if self._n_pool > 0:
109             B_pool = self._cat_pool_basis() # [d_out, r_total]
110             C_pool = self._cat_pool_coeff() # [r_total, d_in]
111             mid = F.linear(x_d, C_pool) # [..., r_total]
112
113             if self.mt_U is not None:
114                 # apply (I + U_mt V_mt^T) to mid
115                 mid = mid + (mid @ self.mt_U) @ self.mt_V.T
116
117             pool_out = F.linear(mid, B_pool)
118
119         return base + self.scaling * (active + pool_out)
120
121     def _cat_pool_basis(self):
122         return torch.cat([self.basis_pool[2 * i] for i in range(self._n_pool)], dim=1)
123
124     def _cat_pool_coeff(self):
125         return torch.cat([self.basis_pool[2 * i + 1] for i in range(self._n_pool)], dim=0)
126
127     def commit_to_pool(self):
128         B_trained = self.lora_B.detach().clone().float() # [d_out, r]
129         A_trained = self.lora_A.detach().clone().float() # [r, d_in]
130
131         # mode="reduced" Q: [d_out, r], R: [r, r]
132         Q, R = torch.linalg.qr(B_trained, mode="reduced")
133
134         # coeff = R @ A, Q @ (R@A) = B @ A
135         RA = R @ A_trained # [r, d_in]
136
137         dtype = self.lora_B.dtype
138         device = self.lora_B.device
139
140         # commitfalse!
141         self.basis_pool.append(
142             nn.Parameter(Q.to(dtype=dtype, device=device), requires_grad=False)
143         )

```

```

144         self.basis_pool.append(
145             nn.Parameter(RA.to(dtype=dtype, device=device), requires_grad=False)
146         )
147         self._n_pool += 1
148
149     def archive_and_freeze_mt(self):
150         U = self.mt_U.detach().clone() if self.mt_U is not None else None
151         V = self.mt_V.detach().clone() if self.mt_V is not None else None
152         self._frozen_mt.append((U, V))
153         # self.mt_U:mt
154         if self.mt_U is not None:
155             self.mt_U.requires_grad_(False)
156             self.mt_V.requires_grad_(False)
157
158     def zero_active_zone(self):
159         with torch.no_grad():
160             self.lora_A.zero_()
161             self.lora_B.zero_()
162
163     @torch.no_grad()
164     def _ortho_grad_hook(self, module, grad_input, grad_output):
165         # project grad_B away from all pool bases to enforce inter-task orthogonality
166         if self.lora_B.grad is None or self._n_pool == 0:
167             return
168         g = self.lora_B.grad.float()
169         for i in range(self._n_pool):
170             b = self.basis_pool[2 * i].detach().float()
171             g = g - b @ (b.T @ g)
172         self.lora_B.grad.copy_(g.to(dtype=self.lora_B.grad.dtype))
173
174     #
175     def mt_block_norms(self):
176         # Frobenius norm of each row-block of M_t; block i corresponds to historical task i+1
177         if self.mt_U is None or self._n_pool == 0:
178             return None
179         M = (self.mt_U @ self.mt_V.T).detach().float()
180         # row represents reuse
181         return [
182             M[i * self.rank:(i + 1) * self.rank, :].norm().item()
183             for i in range(self._n_pool)
184         ]
185
186
187 class OrthoHistManager:
188     def __init__(self, model, target_modules, rank=16, alpha=32, dropout=0.0, mt_rank=8, device=None):
189         self.model = model
190         self.target_modules = tuple(target_modules)
191         self.rank = rank
192         self.alpha = alpha
193         self.mt_rank = mt_rank
194         self.device = device or next(model.parameters()).device
195         self.current_task_id = 0
196         self.layers: Dict[str, OrthoHistLoRALinear] = {}
197     # 1.
198     def inject(self):
199         dtype = next(self.model.parameters()).dtype
200         replaced = {}
201         for name, module in list(self.model.named_modules()):
202             if not isinstance(module, nn.Linear):
203                 continue
204             if not any(name.endswith(t) for t in self.target_modules):

```

```

205         continue
206         parent_name, child_name = name.rsplit(".", 1)
207         parent = self.model.get_submodule(parent_name)
208         wrapper = OrthoHistLoRALinear(
209             module, rank=self.rank, alpha=self.alpha, mt_rank=self.mt_rank
210         ).to(device=self.device, dtype=dtype)
211         setattr(parent, child_name, wrapper)
212         replaced[name] = wrapper
213
214     if not replaced:
215         raise RuntimeError(f"No target modules found. target_modules={self.target_modules}")
216     self.layers = replaced
217
218     # 2. initialization
219     def prepare_for_task(self, task_id):
220         self.current_task_id = task_id
221         for name, layer in self.layers.items():
222             if task_id == 1:
223                 layer._init_active_fresh()
224             else:
225                 layer.init_active_projected()
226                 layer.init_mt()
227         self._set_trainable()
228
229     # 3. set requires grad
230     def _set_trainable(self):
231         for p in self.model.parameters():
232             p.requires_grad_(False)
233         for layer in self.layers.values():
234             layer.lora_A.requires_grad_(True)
235             layer.lora_B.requires_grad_(True)
236             if layer.mt_U is not None:
237                 layer.mt_U.requires_grad_(True)
238                 layer.mt_V.requires_grad_(True)
239
240     # 4.
241     def trainable_parameters(self):
242         params, seen = [], set()
243         for layer in self.layers.values():
244             for p in [layer.lora_A, layer.lora_B, layer.mt_U, layer.mt_V]:
245                 # id(p): parameter's address
246                 if p is not None and p.requires_grad and id(p) not in seen:
247                     params.append(p)
248                     seen.add(id(p))
249         return params
250
251     # 5
252     def commit_task(self):
253         for name, layer in self.layers.items():
254             # commit B and A
255             layer.commit_to_pool()
256             # commit M_t
257             layer.archive_and_freeze_mt()
258             layer.zero_active_zone()
259
260     def n_trainable_params(self):
261         return sum(p.numel() for p in self.trainable_parameters())
262
263     def summary(self):
264         n_pool = next(iter(self.layers.values()))._n_pool
265         return {
266             "method": "ortho_hist",

```

```

266         "task_id": self.current_task_id,
267         "n_pool": n_pool,
268         "r_total": n_pool * self.rank,
269         "mt_rank": self.mt_rank,
270         "trainable_params": self.n_trainable_params(),
271     }
272
273     # 3 Mt commit collect
274     def collect_reuse_analysis(self, task_name):
275         result = {"task_name": task_name, "layers": {}}
276         for name, layer in self.layers.items():
277             norms = layer.mt_block_norms()
278             result["layers"][name] = norms if norms is not None else []
279         return result

```

A.14 Checkpointing Utilities

Listing A.14: src/checkpointing.py: Checkpointing Utilities

```

1  from __future__ import annotations
2
3  import json
4  import os
5  from pathlib import Path
6  from typing import Any, Dict, Optional
7
8  import torch
9
10 def _json_safe(value):
11     if isinstance(value, (str, int, float, bool)) or value is None:
12         return value
13     if isinstance(value, list):
14         return [_json_safe(v) for v in value]
15     if isinstance(value, dict):
16         return {str(k): _json_safe(v) for k, v in value.items()}
17     return str(value)
18
19
20 def _save_json_atomic(payload: Dict[str, Any], path: Path) -> None:
21     tmp = path.with_suffix(path.suffix + ".tmp")
22     with open(tmp, "w", encoding="utf-8") as f:
23         json.dump(_json_safe(payload), f, indent=2, ensure_ascii=False)
24     os.replace(tmp, path)
25
26
27 def save_checkpoint(
28     output_dir,
29     task_id,
30     model,
31     extra: Optional[Dict[str, Any]] = None,
32 ):
33     ckpt_dir = Path(output_dir) / "checkpoints" / f"task_{task_id:02d}"
34     ckpt_dir.mkdir(parents=True, exist_ok=True)
35
36     # Save the full model state. For OrthoHist-LoRA this includes the frozen
37     # historical basis_pool; for Sequential LoRA it includes the PEFT LoRA state.
38     state = model.state_dict()
39     tmp_path = ckpt_dir / "model_state.pt.tmp"

```

```

40     final_path = ckpt_dir / "model_state.pt"
41     torch.save(state, tmp_path)
42     os.replace(tmp_path, final_path)
43
44     if extra:
45         _save_json_atomic(extra, ckpt_dir / "meta.json")
46
47
48 def load_checkpoint(output_dir, task_id, model):
49     ckpt_dir = Path(output_dir) / "checkpoints" / f"task_{task_id:02d}"
50     ckpt_path = ckpt_dir / "model_state.pt"
51
52     if not ckpt_path.exists():
53         raise FileNotFoundError(f"Checkpoint not found: {ckpt_path}")
54
55     state = torch.load(ckpt_path, map_location="cpu")
56     incompatible = model.load_state_dict(state, strict=False)
57
58     missing = list(getattr(incompatible, "missing_keys", []))
59     unexpected = list(getattr(incompatible, "unexpected_keys", []))
60
61     if unexpected:
62         print(f"[Checkpoint WARNING] unexpected keys while loading {ckpt_path}:")
63         for key in unexpected[:20]:
64             print(f" - {key}")
65         if len(unexpected) > 20:
66             print(f" ... and {len(unexpected) - 20} more")
67
68     if missing:
69         print(f"[Checkpoint WARNING] missing keys while loading {ckpt_path}:")
70         for key in missing[:20]:
71             print(f" - {key}")
72         if len(missing) > 20:
73             print(f" ... and {len(missing) - 20} more")
74
75     return incompatible

```

A.15 Analysis and Visualization Utilities

Listing A.15: src/analysis_utils.py: Analysis and Visualization Utilities

```

1  from __future__ import annotations
2
3  import json
4  from pathlib import Path
5
6  import numpy as np
7
8
9  def collect_reuse_matrix(reuse_snapshots, task_names):
10     T = len(task_names)
11     matrix = np.full((T, T), np.nan)
12     task_to_idx = {name: idx for idx, name in enumerate(task_names)}
13
14     for snap in reuse_snapshots:
15         task_name = snap.get("task_name")
16         if task_name not in task_to_idx:
17             continue

```

```

18
19     t_idx = task_to_idx[task_name]
20     if t_idx == 0:
21         continue
22
23     layer_data = snap.get("layers", {})
24     layer_means = []
25     for norms in layer_data.values():
26         if norms and len(norms) == t_idx:
27             layer_means.append(norms)
28
29     if not layer_means:
30         continue
31
32     arr = np.array(layer_means, dtype=float)           # [n_layers, t_idx]
33     mean_per_hist = arr.mean(axis=0)                 # [t_idx]
34     for i, val in enumerate(mean_per_hist):
35         matrix[t_idx, i] = val
36
37     return matrix
38
39
40 def save_reuse_heatmap(matrix, task_names, output_dir):
41     out = Path(output_dir) / "analysis"
42     out.mkdir(parents=True, exist_ok=True)
43
44     serialisable = {
45         task_names[t]: {
46             task_names[i]: float(matrix[t, i])
47             for i in range(t)
48             if not np.isnan(matrix[t, i])
49         }
50         for t in range(len(task_names))
51     }
52
53     with open(out / "reuse_matrix.json", "w", encoding="utf-8") as f:
54         json.dump(serialisable, f, indent=2, ensure_ascii=False)
55
56     try:
57         import matplotlib.pyplot as plt
58
59         fig, ax = plt.subplots(figsize=(9, 7))
60         masked = np.ma.masked_invalid(matrix)
61         im = ax.imshow(masked, aspect="auto", cmap="YlOrRd")
62         plt.colorbar(im, ax=ax, label="Mean M_t block norm (Frobenius)")
63         ax.set_xticks(range(len(task_names)))
64         ax.set_yticks(range(len(task_names)))
65         ax.set_xticklabels(task_names, rotation=45, ha="right", fontsize=8)
66         ax.set_yticklabels(task_names, fontsize=8)
67         ax.set_xlabel("Historical task")
68         ax.set_ylabel("Current task")
69         ax.set_title("M_t reuse strength")
70
71         for t in range(len(task_names)):
72             for i in range(len(task_names)):
73                 v = matrix[t, i]
74                 if not np.isnan(v):
75                     ax.text(i, t, f"{v:.2f}", ha="center", va="center", fontsize=7)
76
77         plt.tight_layout()
78         plt.savefig(out / "reuse_heatmap.png", dpi=150)

```

```

79     plt.close()
80     except Exception as e:
81         print(f"[analysis] could not save reuse heatmap: {e}")
82
83
84 def save_score_matrix(score_matrix, task_names, output_dir):
85     out = Path(output_dir) / "analysis"
86     out.mkdir(parents=True, exist_ok=True)
87
88     with open(out / "score_matrix.json", "w", encoding="utf-8") as f:
89         json.dump(score_matrix, f, indent=2, ensure_ascii=False)
90
91     try:
92         import matplotlib.pyplot as plt
93
94         T = len(task_names)
95         mat = np.full((T, T), np.nan)
96
97         for r_idx, r_name in enumerate(task_names):
98             row = score_matrix.get(r_name, {})
99             for c_idx, c_name in enumerate(task_names):
100                 if c_name in row:
101                     mat[r_idx, c_idx] = row[c_name]
102
103         fig, ax = plt.subplots(figsize=(9, 7))
104         im = ax.imshow(np.ma.masked_invalid(mat), aspect="auto", cmap="Blues", vmin=0, vmax=1)
105         plt.colorbar(im, ax=ax, label="Score")
106         ax.set_xticks(range(T))
107         ax.set_yticks(range(T))
108         ax.set_xticklabels(task_names, rotation=45, ha="right", fontsize=8)
109         ax.set_yticklabels(task_names, fontsize=8)
110         ax.set_xlabel("Evaluated task")
111         ax.set_ylabel("Trained up to task")
112         ax.set_title("Continual learning score matrix")
113
114         for r in range(T):
115             for c in range(T):
116                 v = mat[r, c]
117                 if not np.isnan(v):
118                     ax.text(c, r, f"{v:.2f}", ha="center", va="center", fontsize=7)
119
120         plt.tight_layout()
121         plt.savefig(out / "score_matrix.png", dpi=150)
122         plt.close()
123     except Exception as e:
124         print(f"[analysis] could not save score matrix: {e}")
125
126
127 def count_trainable_params(model):
128     trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
129     total = sum(p.numel() for p in model.parameters())
130     return {"trainable": trainable, "total": total}

```


Appendix B

Appendix B: User Manual

B.1 Quick Start

Listing B.1: Quick start.

```
1 git clone PUT-GITHUB-REPOSITORY-URL-HERE
2 cd ECE-5996-design-proj-main
3 pip install -r requirements.txt
4 python main.py --config configs/sequential_lora.yaml
5 python main.py --config configs/ortho_hist_k4.yaml
```

B.2 Reading Final Metrics

Listing B.2: Read final metrics from a run.

```
1 import json
2 from pathlib import Path
3
4 run = Path("outputs/ortho_hist_k4")
5 hist = json.load(open(run / "history.json", "r", encoding="utf-8"))
6 last = hist[-1]
7 print(last["average_accuracy"])
8 print(last["bwt"])
9 print(last["seen_scores"])
```